

Multilinear Tensor Low-Rank Approximation for Policy-Gradient Methods in Reinforcement Learning

Sergio Rozada , *Student Member, IEEE*, Hoi-To Wai , *Member, IEEE*,
and Antonio G. Marques , *Senior Member, IEEE*

Abstract—Reinforcement learning (RL) aims to estimate the action to take given a (time-varying) state, with the goal of maximizing a cumulative reward function. Predominantly, there are two families of algorithms to solve RL problems: value-based and policy-based methods, with the latter designed to learn a probabilistic parametric policy from states to actions. Most contemporary approaches implement this policy using a neural network (NN). However, NNs usually face issues related to convergence, architectural suitability, hyper-parameter selection, and underutilization of the redundancies of the state-action representations (e.g. locally similar states). This paper postulates multilinear mappings to efficiently estimate the parameters of the RL policy. More precisely, we leverage the PARAFAC decomposition to design *tensor low-rank* policies. The key idea involves collecting the policy parameters into a tensor and leveraging tensor-completion techniques to enforce *low rank*. We establish theoretical guarantees of the proposed methods for various policy classes and validate their efficacy through numerical experiments. Specifically, we demonstrate that *tensor low-rank* policy models reduce computational and sample complexities in comparison to NN models while achieving similar rewards.

Index Terms—Reinforcement learning, policy gradients, trust-region methods, low-rank approximation, tensors.

I. INTRODUCTION

WE inhabit a world characterized by growing complexity and interdependence, where gadgets produce vast streams of data on a daily basis. Computer systems need to become intelligent to make decisions autonomously in order to navigate such complex landscapes with little human supervision. Among the various frameworks suggested for developing

autonomous intelligent systems, Reinforcement Learning (RL) has emerged as particularly promising. Broadly speaking, software agents in RL learn by interacting with the world by trial and error [1], [2]. RL has contributed significantly to pivotal achievements within the Artificial Intelligence (AI) community. Notable examples of the latter include AlphaGo, a computer software that beat the world champion in the game of Go [3], [4], and recently, groundbreaking chatbots like ChatGPT [5].

From a technical standpoint, RL is rooted in stochastic approximation for Dynamic Programming (DP). RL solves a sequential optimization problem, aiming to maximize a reward signal over time [6], [7]. In this framework, the world, or environment, is typically represented as a Markov Decision Process (MDP) [1]. Succinctly, the environment consists of a set of states, or state space, where the agents take actions from a set of actions, or action space. Notably, the states and the actions are usually multi-dimensional. Unlike DP, RL does not assume knowledge of the underlying dynamics of the MDP model, with learning taking place from sampled trajectories. In a nutshell, the goal of RL is to learn a *state-action map*, or *policy*, that maximizes the expected cumulative reward from samples of the MDP. Originally, value-based methods proposed to learn the optimal value functions (VFs) within the MDP first, to then infer the optimal policy [1], [2]. Yet, value-based methods face problems due to the curse of dimensionality. An alternative to overcome this limitation is offered by policy-based RL, which proposes to learn directly a policy, which almost always is assumed to be probabilistic and parametric [8]. Therefore, the problem shifts from learning the VFs to estimate the parameters of a probabilistic mapping. Typically, Gaussian distributions are employed for continuous action spaces, and softmax distributions for discrete action spaces [9], [10], [11].

The conventional approach to solve policy-based RL problems involves learning a mapping from states to the *parameters of the probability distribution* via gradient-based algorithms, also known as policy-gradient methods (PGs) [12]. Designing policy maps involves balancing expressivity and tractability. Initially, linear models were favored for their simplicity and ease of understanding, but they lacked expressive power and required significant feature engineering. More recently, neural networks (NNs) have emerged as an alternative to handle more complex mapping tasks. However, finding the adequate NN architecture is often a challenge on its own. A poor selection of

Received 10 January 2025; revised 15 September 2025 and 27 October 2025; accepted 17 November 2025. Date of publication 24 November 2025; date of current version 16 December 2025. This work was supported in part by the Spanish NSF under Grant MCIN/AEI/10.13039/501100011033, Grant PID2019-105032GB-I00, Grant TED2021-130347B-I00, and Grant PID2022-136887NB-I00, in part by the European Union NextGenerationEU/PRTR, and in part by the Community of Madrid via the Ellis Madrid Unit under Grant TEC-2024/COM-89. The associate editor coordinating the review of this article and approving it for publication was George Atia. (*Corresponding author: Sergio Rozada.*)

Sergio Rozada and Antonio G. Marques are with the Department of Signal Theory and Comms., King Juan Carlos University, 28942 Madrid, Spain (e-mail: s.rozada.2019@alumnos.urjc.es; antonio.garcia.marques@urjc.es).

Hoi-To Wai is with the Department of Systems Engineering and Engineering Management, The Chinese University of Hong Kong (CUHK), SAR 999077, Hong Kong, China (e-mail: htwai@cuhk.edu.hk).

Digital Object Identifier 10.1109/TSP.2025.3636071

the architecture may lead to convergence issues and moreover, even when the architecture seems fitting, NN training can be time-consuming. To narrow the space of potential models and better condition the problem, a practical strategy is to postulate parsimonious models that exploit the problem's inherent structure. Sparsity, in particular, has been widely leveraged to design policy-based algorithms [13], [14], [15]. More recently, low-rank methods have been explored in value-based RL by enforcing low-rank structure in matrix and tensor representations of VFs [16], [17], [18]. Intriguingly, however, low-rank models for policy-based RL remain largely unexplored [19]. This gap is the main driver of this paper, that puts forth low-rank policy models that give rise to *efficient* and *fast* policy-based RL algorithms. Given the potentially high dimensionality of the state space, we propose to collect the state-dependent parameters of the policy probability distribution in a tensor of parameters, to then enforce low rank via tensor completion techniques. This gives rise to multilinear *tensor low-rank* policy models can be seamlessly integrated into classical policy-based RL algorithms, as we will show in the following sections.

Low-rank methods have been widely applied in optimization, most notably in the design of matrix completion algorithms [20], [21], [22]. In the realm of tensors, the multidimensional generalization of matrices, low-rank techniques have also proven successful in developing resource-efficient optimization methods [23], [24]. Interestingly, MDPs can also exhibit low-rank structure, meaning that both transition and reward dynamics are low rank [25], [26]. Leveraging this property has led to the development of efficient model-based algorithms, where enforcing low rank in matrix or tensor representations of transition dynamics has been instrumental [27], [28], [29], [30]. Moreover, recent empirical studies have revealed that matrix and tensor representations of VFs are often low rank [16], [18]. This observation has motivated value-based methods that reduce sample complexity [31], [32] and demonstrate faster empirical convergence rates [16], [17], [18], [33]. In contrast, the exploration of low-rank methods in policy-based RL has been more limited. Recent work has introduced matrix-completion techniques to design Gaussian policy models in actor-critic (AC) methods and in trust-region policy optimization (TRPO) [19], [34]. However, matrix-based policies do not scale well to large state-action spaces, and since action spaces are not always continuous, Gaussian policies can often be a poor choice.

Regarding convergence, policy-gradient algorithms have been extensively studied to understand their properties. Global convergence has been established for tabular softmax policies, where each state-action pair has its own parameter and the policy is obtained by applying a softmax over them, assuming exact gradient estimates [35], [36] or (nearly) unbiased stochastic approximations [37], [38]. For more general function classes, such as NNs, global convergence can still be proven under the Polyak-Łojasiewicz (PL) condition [35], [39], although this assumption is often restrictive in practice. In more general scenarios, sublinear convergence to local optima can be guaranteed by assuming smooth objectives and Lipschitz-continuous gradients [40]. However, these guarantees

require unbiased gradient estimates, and NNs might violate gradient Lipschitz-continuity due to nonlinear activations, e.g., ReLU [41]. In contrast, tensor-based models are multilinear, which facilitates proving smoothness and Lipschitz-continuity of their gradients. Moreover, their low-rank structure can reduce the associated Lipschitz constants, leading to tighter convergence bounds.

Motivated by these discoveries, this paper aims to present a comprehensive methodology for crafting low-rank policy-based RL algorithms. Our specific contributions include:

- C1** Policy *models* that enforce low rank in a tensor of Gaussian-policy parameters and softmax-policy parameters using the PARAFAC decomposition. This is equivalent to postulate a multilinear model for the policy.
- C2** Policy-based *algorithms*, including REINFORCE, AC, TRPO and PPO, leveraging tensor low-rank (multilinear) policy models.
- C3** *Convergence guarantees* showing that the proposed tensor low-rank policy gradient algorithms achieve sublinear convergence rates to local optima, with the convergence neighborhood shrinking as the rank of the policy tensor decreases.
- C4** *Numerical experiments* illustrating that the proposed low-rank methods converge faster than NN-based schemes.

II. PRELIMINARIES

The objective of this section is to introduce notation, cover the basics of RL, and provide a concise overview of tensor low-rank methodologies.

A. Reinforcement Learning

RL is a framework to solve sequential optimization problems in Markovian setups [1], [2]. Not surprisingly, the main mathematical formalism is the Markov Decision Process (MDP), which defines the environment as a set of states $\mathcal{S}$ and a set of actions $\mathcal{A}$. The dynamics of the MDP are encoded in the probability transition function $p(s'|s, a)$, which defines the probability of transitioning from state $s \in \mathcal{S}$ to state $s' \in \mathcal{S}$ after taking action $a \in \mathcal{A}$. The initial state is distributed according to p_1 . The reward function $r(s, a, s')$ defines the instantaneous reward associated with being in the state $s \in \mathcal{S}$, taking the action $a \in \mathcal{A}$, and reaching the new state $s' \in \mathcal{S}$. Interestingly, the reward function r can be deterministic or stochastic. A key aspect of RL is that we do not have access to p , p_1 , and r , often referred to as the model of the MDP.

The fundamental goal of RL is to learn the optimal sequence of actions to maximize the long-termed reward. In policy-based RL we aim to learn a probabilistic parametric policy π_θ that maximizes the cumulative reward over a period of T time steps. Let $\theta \in \mathbb{R}^p$ be a policy parameter vector, the policy $\pi_\theta(a|s)$ is a map that defines the conditional probability of taking the action $a \in \mathcal{A}$ being in state $s \in \mathcal{S}$. As the model of the MDP is unknown, the policy π_θ is learned from trajectories sampled from the MDP. Let $t = 1, \dots, T$ be a time index. The trajectory $\tau_T = \{(S_t, A_t)\}_{t=1}^{T+1} \sim g(\theta)$ is a Markov chain generated by $S_1 \sim p_1$, and for any $t \geq 1$, $A_t \sim \pi_\theta(\cdot|S_t)$, $S_{t+1} \sim p(\cdot|S_t, A_t)$.

The reward $R_t = r(S_t, A_t, S_{t+1})$ is a random variable that evaluates the *immediate* value of the selected state-action pair at time-step t . However, the action A_t influences $S_{t'}$ for $t' > t$, thereby impacting subsequent rewards $R_{t'}$ for $t' > t$. This illustrates that the problem is coupled across time and the cumulative reward, or return, of a trajectory $G_T(\tau_T) = \sum_{t=1}^T R_t$ must be considered in the optimization.

Subsequently, the policy optimization problem can be written as

$$\max_{\theta \in \mathbb{R}^p} \bar{G}_V(\theta) := \mathbb{E}_{\tau_T \sim g(\theta)} [G_T(\tau_T)], \quad (1)$$

where the decision variable appears in the distribution that leads to the sampled trajectory τ_T and the problem is non-convex in general. To see this clearer, consider a simple setup involving actions modeled as samples of a univariate Gaussian policy as $A_t \sim \mathcal{N}(\mu_{S_t}, \sigma_{S_t})$, with policy parameters $\theta_\mu = \{(\mu_s)\}_{s \in \mathcal{S}}$ and $\theta_\sigma = \{(\sigma_s)\}_{s \in \mathcal{S}}$. Clearly, the dimension of θ depends on the state space $\mathcal{S}$ that can become infinity for continuous state space. The dimensionality problem is usually addressed via parametric mappings that estimate the parameters of the probability distribution associated with the policy, e.g. $\mu_{\theta_\mu}, \sigma_{\theta_\sigma} : \mathcal{S} \mapsto \mathbb{R}$ in the Gaussian case. The problem (1) evaluates the expected returns over trajectories, that are collections of transitions. For each transition, at a given state s , the action a is sampled from the Gaussian distribution $\mathcal{N}(\mu_s, \sigma_s)$, where the mean and the standard deviations are given by $\mu_s := \mu_{\theta_\mu}(s)$ and $\sigma_s := \sigma_{\theta_\sigma}(s)$. Next, we overview several common strategies for optimizing (1).

Policy-gradient methods. Directly optimizing (1) entails using the density function $g(\theta)$ to compute an expectation over the trajectories. However, $g(\theta)$ relies on the unknown probability transition function p and the initial state distribution p_1 . As introduced previously, stochastic approximation methods are commonly used to circumvent this issue by approximating the expectation using samples $\tau_T \sim g(\theta)$ from the MDP. The workhorse of policy-based RL is the gradient-based algorithms that update the policy parameter vector θ through (stochastic) gradient ascent [42]. Consequently, the update rule takes the form $\theta^{h+1} = \theta^h + \eta^h \nabla_\theta \bar{G}_V(\theta^h)$, where h denotes the iteration index, and η^h represents the step size or learning rate. The policy gradient theorem [8] shows that the gradient of $\bar{G}_V(\theta)$ can be evaluated as

$$\nabla_\theta \bar{G}_V(\theta) = \mathbb{E}_{g(\theta)} \left[\sum_{t=1}^T G_T(\tau_T) \nabla_\theta \log \pi_\theta(A_t | S_t) \right]. \quad (2)$$

Interestingly, these schemes receive the name of policy gradient (PG) methods, as computing the gradient of $\bar{G}_V$ boils down to computing the gradient of the log-policy $\nabla_\theta \log \pi_\theta$ with respect to (w.r.t.) the parameters θ . The term $\nabla_\theta \log \pi_\theta$ is also known as the *policy score*. A celebrated PG method is Monte-Carlo PG (a.k.a. REINFORCE) [43], which samples trajectories from the MDP to update the parameters via (2), using the sample return $G_t = \sum_{t'=t}^T R_{t'}$.

Actor-critic methods. Monte-Carlo PG methods suffer from the credit assignment problem, i.e., determining the contribution of each action becomes challenging when the return is

observed only at the end of the episode. This often leads to high-variance issues [44]. A common strategy to mitigate the high variance in the gradient estimate while maintaining the bias unchanged involves subtracting a baseline value from the return G_t [45]. The most common baseline is the VF, defined as $\mathbb{V}^{\pi_\theta}(S_t) = \mathbb{E}_{\pi_\theta} [G_t | S_t]$. This results in an estimator of the advantage function [44] as $\mathbb{A}^{\pi_\theta}(S_t) = G_t - \mathbb{V}^{\pi_\theta}(S_t)$. However, computing the exact VFs associated with the policy π_θ is a challenge by itself, usually addressed by learning a parametric approximation $\mathbb{V}_\omega$. This leads to AC methods, that operate in two phases, by estimating first the parameters of the policy π_θ or actor, and then the parameters of the VF approximator $\mathbb{V}_\omega$, or critic [46]. The AC counterpart of (2) is

$$\nabla_\theta \bar{G}_A(\theta) = \mathbb{E}_{g(\theta)} \left[\sum_{t=1}^T \mathbb{A}_\omega(S_t) \nabla_\theta \log \pi_\theta(A_t | S_t) \right], \quad (3)$$

where $\mathbb{A}_\omega(S_t) = G_t - \mathbb{V}_\omega(S_t)$ is the advantage estimator, with $\mathbb{V}_\omega(S_t)$ fixed. Stochastic gradient updates of the actor reduces again to compute the policy scores $\nabla_\theta \log \pi_\theta$. The critic parameters ω are obtained via the minimization of the mean-squared error $L(\omega) = \frac{1}{2} \sum_{t=1}^T (G_t - \mathbb{V}_\omega(S_t))^2$, typically using gradient descent methods of the form $\omega^{h+1} = \omega^h - \alpha^h \nabla_\omega L(\omega^h)$, where h is again the iteration index and α^h is the gradient step.

Trust-region methods. Interestingly, monotonic improvements of the policy can be guaranteed by constraining the policy updates within small trust-regions [47], [48], [49]. This fact motivated trust-region policy optimization (TRPO) [49], a method that proposes reformulating the actor update in the AC problem to find a new policy π_θ in a trust-region of the actual (old) policy $\pi_{\bar{\theta}}$ used to sample from the environment. This region is defined by the KL-divergence $D_{KL}(\cdot \| \cdot)$. More formally, we denote the probability ratio between the new policy π_θ and the (old) policy $\pi_{\bar{\theta}}$ as

$$q_{\theta, \bar{\theta}}(S_t, A_t) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\bar{\theta}}(A_t | S_t)}. \quad (4)$$

Then, TRPO proposes to iteratively maximize:

$$\bar{G}_T(\theta) = \mathbb{E}_{g(\bar{\theta})} \left[\sum_{t=1}^T q_{\theta, \bar{\theta}}(S_t, A_t) \mathbb{A}_\omega(S_t) \right], \quad (5)$$

subject to $\mathbb{E} [D_{KL}(\pi_{\bar{\theta}}(\cdot | s) \| \pi_\theta(\cdot | s))] \leq \delta$, where $\mathbb{E} [\cdot]$ denotes the expectation over the states, and δ is the size of the neighborhood. TRPO involves i) sampling the MDP following $\pi_{\bar{\theta}}$ to approximate the expectation $\mathbb{E}_{g(\bar{\theta})} [\cdot]$, and ii) maximize (5) using the samples, usually via second order optimization methods [49]. In setups where second-order methods are computationally demanding, proximal policy optimization (PPO) [50] proposes constraining the policy updates using a clipping mechanism to design first-order TR methods. More specifically, PPO defines a trust-region around the actual (old) policy $\pi_{\bar{\theta}}$ using the clipped probability ratio

$$\hat{q}_{\theta, \bar{\theta}}(S_t, A_t) = \begin{cases} 1 - \epsilon & \text{if } q_{\theta, \bar{\theta}}(S_t, A_t) \leq 1 - \epsilon \\ 1 + \epsilon & \text{if } q_{\theta, \bar{\theta}}(S_t, A_t) \geq 1 + \epsilon \\ q_{\theta, \bar{\theta}}(S_t, A_t) & \text{otherwise,} \end{cases} \quad (6)$$

where ϵ defines the size of the trust-region, to then maximize the following objective

$$\bar{G}_P(\theta) = \mathbb{E}_{g(\bar{\theta})} \left[\sum_{t=1}^T \min \{ q_{\theta, \bar{\theta}}(S_t, A_t) \mathbb{A}_\omega(S_t), \hat{q}_{\theta, \bar{\theta}}(S_t, A_t) \mathbb{A}_\omega(S_t) \} \right]. \quad (7)$$

Note that the objective in (7) clips the probability ratio to $1 + \epsilon$ for positive advantages and to $1 - \epsilon$ for negative advantages. As we will show in subsequent sections, the actor update in TR methods (TRPO and PPO) boils down to compute the policy scores $\nabla_\theta \log \pi_\theta$, as in PG and AC methods. This is a fundamental aspect of policy-based algorithms, since it implies that postulating new policy models can be “decoupled” from the algorithmic design. For the problems investigated in this paper, this implies that the *low-rank multilinear* policy models that we introduce in subsequent sections can be applied to different policy-based RL algorithms, provided that those algorithms are modified to use the proposed policy scores.

B. Low Rank and Tensor Completion

Since tensors are a central piece of this work, we briefly review related basic concepts and notation. In data science contexts, tensors can be understood as a multi-dimensional generalization of matrices. From a formal standpoint, a tensor $\underline{\mathbf{X}} \in \mathbb{R}^{N_1 \times \dots \times N_D}$ is a D -dimensional array indexed by a D -dimensional index $\mathbf{i} = [i_1, \dots, i_D]$ with $i_d \in \{1, \dots, N_d\}$. In tensor jargon, dimensions also receive the name of modes. In contrast to matrices, there are different definitions for the rank of the tensor. In this work, we focus on the PARAFAC rank, which has been widely used to postulate parsimonious data representations. Succinctly, the PARAFAC rank is defined as the minimum number of rank-1 tensors that, when aggregated, recover the original tensor [23], [24]. To be more precise, a tensor $\underline{\mathbf{Z}}$ has rank-1 if it can be written as the outer product of D vectors. This is denoted as $\underline{\mathbf{Z}} = \mathbf{z}_1 \odot \dots \odot \mathbf{z}_D$ and implies that the entry $\mathbf{i}$ of rank-1 tensor is simply given by $[\underline{\mathbf{Z}}]_{\mathbf{i}} = \prod_{d=1}^D [\mathbf{z}_d]_{i_d}$. Then, $\underline{\mathbf{X}}$ has a rank K if it can be decomposed as $\underline{\mathbf{X}} = \sum_{k=1}^K \mathbf{x}_1^k \odot \dots \odot \mathbf{x}_D^k$, with the PARAFAC decomposition [51] being the procedure that, given $\underline{\mathbf{X}}$, finds the vectors $\{\mathbf{x}_d^k\}_{d=1, k=1}^{D, K}$, known as factors. The factor vectors are oftentimes collected into matrices $\mathbf{X}_d = [\mathbf{x}_d^1, \dots, \mathbf{x}_d^K]$. Using the entry-wise definition of the outer product, it readily follows that entry of a tensor of rank K is given by

$$[\underline{\mathbf{X}}]_{\mathbf{i}} = \sum_{k=1}^K \prod_{d=1}^D [\mathbf{x}_d^k]_{i_d} = \sum_{k=1}^K \prod_{d=1}^D [\mathbf{X}_d]_{i_d, k}, \quad (8)$$

revealing that assuming that the tensor has low-rank implies modeling the tensor as a multilinear combination of the factors. The equation above illustrates that the PARAFAC decomposition is a parsimonious representation of the original tensor. The original tensor has $\prod_{d=1}^D N_d$ elements and its size grows exponentially with the number of dimensions. On the other hand, the number of degrees of freedom under the PARAFAC representation is $\left(\sum_{d=1}^D N_d \right) K$, and more importantly, it grows

additively with the number of dimensions. Notably, the only (hyper-)parameter that needs tuning in the PARAFAC model defined in (8) is the rank K , which will emerge as one of the advantages of the policy models we will propose in the following section.

III. TENSOR LOW-RANK FOR PG METHODS

This section contains the main contribution of the paper. It starts by introducing *tensor low-rank tensor* policy models for general classes of policies, and specifically for Gaussian and softmax policies. We then use the tensor low-rank policies to design *tensor low-rank* PG, AC and TR algorithms to solve policy-based RL problems. Furthermore, we show that under suitable conditions, the PG algorithm converges to a stationary point of (1).

A. Tensor Policy Models

When designing parametric functional estimators, it is necessary to trade-off expressiveness and number of parameters. This is essential as we seek a meaningful approximation while taking care of the convergence time and the required sample size. To address this challenge in RL policy design, we propose using multilinear policy models that leverage the fact that many MDPs admit lower-dimensional representations [16], [18], [25]. As introduced in the previous section, the design of policy-based RL algorithms with alternative policy parametrizations boils down to postulating meaningful models, and characterizing the policy scores $\nabla_\theta \log \pi_\theta$ to tune the parameters of the model. Hence, in this section we introduce the tensor low-rank policy model and derive the associated policy scores. When the policy scores are easy to evaluate, the associated policy model can be readily deployed in a wide variety of policy-based algorithms. The policy scores depend on i) the probability distribution, and ii) the functional form of the parametrization θ . To illustrate this, consider the example of a one-dimensional continuous-action space, where actions are modeled as samples from a univariate Gaussian policy. As introduced earlier, the mean and standard deviation are typically parameterized by the mappings μ_{θ_μ} and σ_{θ_σ} . NNs are commonly employed to implement these mappings, resulting policy gradients $\nabla_\theta \log \pi_\theta$ with closed-form that depend on the Gaussian probability density function and the NN architecture. While this approach yields tractable expressions that are straightforward to compute, it presents several challenges: (i) convergence issues, (ii) susceptibility to overfitting and overparameterization, and (iii) a failure to exploit the fact that many MDPs can be effectively represented in lower-dimensional spaces [16], [18], [25], [52]. The first contribution of this paper puts forth a parsimonious multilinear policy model that involves rearranging the set θ into a multi-dimensional tensor to then leverage *low rank* via the PARAFAC decomposition.

Let $\mathcal{S} = \mathcal{S}_1 \times \dots \times \mathcal{S}_D$ denote the D -dimensional state space. The state space is assumed to be discrete and the cardinality of $\mathcal{S}_d$ is denoted as N_d . Without loss of generality, consider a generic probabilistic policy with parameters $\{\theta_s\}_{s \in \mathcal{S}}$ where, for simplicity, θ_s denotes the (one-dimensional)

parameter of a probability distribution. We propose rearranging the parameters as a tensor $\underline{\Theta} \in \mathbb{R}^{N_1 \times \dots \times N_D}$. Let $\mathbf{i}_s = [i_1^s, \dots, i_D^s]$ denote the index of the tensor associated with the state s , then the parameter θ_s associated with the state s can be retrieved from the tensor as $\theta_s = [\underline{\Theta}]_{\mathbf{i}_s}$. However, the mere rearrangement of the parameters into a tensor format does not portray any benefits per se, as the number of parameters of the tensor grows exponentially with D . In order to alleviate this problem, we postulate that the tensor $\underline{\Theta}$ is low rank with rank K . Furthermore, we leverage the PARAFAC decomposition in (8) that parameterize the tensor through the collection of D factors of rank K , i.e., $\Theta = \{\Theta_d\}_{d=1}^D$, where $\Theta_d \in \mathbb{R}^{N_d \times K}$. This yields the low-rank policy optimization problem:

$$\max_{\Theta} \bar{G}_V(\underline{\Theta}) \text{ s.t. } [\underline{\Theta}]_{\mathbf{i}_s} = \sum_{k=1}^K \prod_{d=1}^D [\Theta_d]_{i_d^s, k}. \quad (9)$$

Under the tensor low-rank framework, actions are sampled from probability distributions whose parameters are represented as a multilinear model directly given by the factors Θ . Thus, the optimization problem (9) seeks to determine the factors that maximize the cumulative reward. Note that the original tensor $\underline{\Theta}$ has $\prod_{i=1}^D N_i$ parameters, and grows exponentially with the number of dimensions D . In contrast, the low-rank model parameterized by the factors Θ has $\sum_{i=1}^D N_i K$ parameters, growing only linearly with D . As a remark, one can trivially extend the tensor of parameters to accommodate probability distributions with multiple parameters by considering an additional dimension in the tensor, as we will show later.

We can now define the policy scores of the tensor policy. The policy scores are the vectorized partial derivatives of the log-policy w.r.t. all the entries of the factors Θ

$$\nabla_{\Theta} \log \pi_{\Theta}(a|s) = \begin{bmatrix} \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\Theta_1]_{1,1}} \\ \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\Theta_1]_{1,2}} \\ \vdots \\ \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\Theta_D]_{N_D, K}} \end{bmatrix}. \quad (10)$$

Then, the partial derivative of $\log \pi_{\Theta}(a|s)$ w.r.t. the (i, k) th entry of Θ_d can be factorized using the chain rule as

$$\begin{aligned} \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\Theta_d]_{i,k}} &= \sum_{\mathbf{i}_s \in S} \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\underline{\Theta}]_{\mathbf{i}_s}} \frac{\partial [\underline{\Theta}]_{\mathbf{i}_s}}{\partial [\Theta_d]_{i,k}} \\ &= \sum_{\mathbf{i}_s \in S} \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\underline{\Theta}]_{\mathbf{i}_s}} \mathcal{I}_{\mathbf{i}_s = \mathbf{i}_s} \frac{\partial [\underline{\Theta}]_{\mathbf{i}_s}}{\partial [\Theta_d]_{i,k}} \\ &= \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\underline{\Theta}]_{\mathbf{i}_s}} \frac{\partial [\underline{\Theta}]_{\mathbf{i}_s}}{\partial [\Theta_d]_{i,k}} \\ &= \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\underline{\Theta}]_{\mathbf{i}_s}} \left(\prod_{\substack{j=1 \\ j \neq d}}^D [\Theta_j]_{i_j^s, k} \right) \mathcal{I}_{i=i_d^s}. \end{aligned} \quad (11)$$

where $\mathcal{I}$ is the indicator function, which takes the value 1 when the condition of the subscript is met and 0 otherwise. Hence, the expression in (11) states that the derivative is only nonzero if $i = i_d^s$ and, as a result, only KD of the entries in (10) are nonzero.

Interestingly, the final expression in (11) factorizes the entry-wise policy score into two terms. The former depends on the specific probability distribution, while the latter is determined by the (tensor) model. Next, we derive the specific form of (11) for two widely-adopted probability distributions in policy-based RL: Gaussian policies for continuous action spaces, and softmax policies for discrete action spaces.

Gaussian policies. As introduced previously, in setups with a continuous one-dimensional action space $\mathcal{A}$, it is common to model actions as samples of the univariate Gaussian distribution $A_t \sim \mathcal{N}(\mu_{S_t}, \sigma_{S_t})$. Practical implementations often consider $\sigma(S_t)$ to be constant and independent of the state to encourage exploration. Then, we focus on the estimation of μ_{S_t} and propose rearranging the set of means $\theta_{\mu} = \{\mu_s\}_{s \in S}$ into the tensor $\underline{\Theta}_{\mu} \in \mathbb{R}^{N_1 \times \dots \times N_D}$, so that $\mu_s = [\underline{\Theta}_{\mu}]_{\mathbf{i}_s}$. We then postulate that $\underline{\Theta}_{\mu}$ has rank K , with factors $\Theta_{\mu} = \{\Theta_{\mu}^d\}_{d=1}^D$. As a result, the entry-wise policy scores defined in (11) for the Gaussian case are

$$\frac{\partial \log \pi_{\Theta_{\mu}}(a|s)}{\partial [\Theta_{\mu}^d]_{i,k}} = \frac{a - [\underline{\Theta}_{\mu}]_{\mathbf{i}_s}}{\sigma^2} \left(\prod_{j=1, j \neq d}^D [\Theta_{\mu}^j]_{i_j^s, k} \right) \mathcal{I}_{i=i_d^s}. \quad (12)$$

Softmax policies. Consider now a one-dimensional discrete-action space $\mathcal{A} = \{1, 2, \dots, C\}$. Actions can be modeled as samples of the probability distribution defined by the softmax function as $A_t \sim \mathcal{Z}(\mathbf{z}_{S_t}, \beta)$, where $\mathbf{z}_{S_t}$ are a set of logits associated with the state S_t , and β is a temperature parameter to promote exploratory actions in the initial phases of learning. The softmax function $\mathcal{Z}: \mathbb{R}^C \mapsto [0, 1]^C$ is a vector function that maps $\mathbf{z} = [z_1, \dots, z_C]^T$ into a probability distribution, with its i -th entry being defined as $\mathcal{Z}(\mathbf{z}, \beta)_i = \frac{e^{\beta z_i}}{\sum_{j=1}^C e^{\beta z_j}}$. We can see the logits $\mathbf{z}_s$ as a fiber of a tensor $\underline{\Theta}_{\mathbf{z}} \in \mathbb{R}^{N_1 \times \dots \times N_D \times C}$. In this case, $\underline{\Theta}_{\mathbf{z}}$ has an extra dimension which is indexed with the actions. Then, with a slightly abuse of notation we define $\mathbf{i}_s = [i_1^s, \dots, i_D^s]$ and $\mathbf{i}_{s,a} = [i_1^{s,a}, \dots, i_{D+1}^{s,a}] = [\mathbf{i}_s; a]$ to index the fiber $\mathbf{z}_s = [\underline{\Theta}_{\mathbf{z}}]_{\mathbf{i}_s}$ and the entry $z_{s,a} = [\underline{\Theta}_{\mathbf{z}}]_{\mathbf{i}_{s,a}}$ respectively. As before, we postulate that $\underline{\Theta}_{\mathbf{z}}$ is a K -rank tensor, and under the PARAFAC model the factors are $\Theta_{\mathbf{z}} = \{\Theta_{\mathbf{z}}^d\}_{d=1}^{D+1}$. In the softmax case the entry-wise policy-scores in (11) are somewhat more complicated, since we have to take derivatives w.r.t. the fiber of $\underline{\Theta}_{\mathbf{z}}$ corresponding to the index $\mathbf{i}_s$, leading to

$$\begin{aligned} \frac{\partial \log \pi_{\Theta_{\mathbf{z}}}(a|s)}{\partial [\Theta_{\mathbf{z}}^d]_{i,k}} &= \sum_{b=1}^C \beta (\mathcal{I}_{a=b} - \mathcal{Z}([\underline{\Theta}_{\mathbf{z}}]_{\mathbf{i}_s}, \beta)_b) \\ &\quad \left(\prod_{j=1, j \neq d}^{D+1} [\Theta_{\mathbf{z}}^j]_{i_j^{s,b}, k} \right) \mathcal{I}_{i=i_d^{s,b}}, \end{aligned} \quad (13)$$

where, again, the partial derivatives of $\log \pi_{\Theta_{\mathbf{z}}}(a|s)$ w.r.t. other fibers $\mathbf{i} \neq \mathbf{i}_s$ are zero.

Sampling the MDP. As explained in the previous section, since the dynamics of the MDP are unknown, RL learns by interacting with the environment. A common mechanism underlying various algorithms discussed subsequently involves sampling trajectories from the MDP. Specifically, we sample a set of trajectories $\{\tau_u\}_{u=1}^U \sim g(\Theta_{\mu})$ following the Gaussian policy $\pi_{\Theta_{\mu}}$ for continuous actions or $\{\tau_u\}_{u=1}^U \sim g(\Theta_{\mathbf{z}})$

Algorithm 1 Tensor Low-Rank Gaussian-Policy Sampling

Require: Policy and factors Θ_μ ; fixed standard deviation σ ; number of trajectories per sample U ; and number of time-steps per trajectory T .

```

function GAUSSIAN_SAMPLING( $\Theta_\mu, U, T, \sigma$ )
  for  $u = 1, \dots, U$  do
    Observe  $S_1^u$ 
    for  $t = 1, \dots, T$  do
       $\mu_{S_t^u} \leftarrow \sum_{k=1}^K \prod_{d=1}^D [\Theta_d^\mu]_{i_d^{S_t^u}, k}$ 
       $A_t^u \sim \mathcal{N}(\mu_{S_t^u}, \sigma)$ 
      Take  $A_t^u$ , observe  $S_{t+1}^u$ , and  $R_t^u$ 
       $S_t^u \leftarrow S_{t+1}^u$ 
    end for
     $\tau_u = \{(S_t^u, A_t^u)\}_{t=1}^T$ 
  end for
  return  $\{\tau_u\}_{u=1}^U$ 
end function

```

Algorithm 2 Tensor Low-Rank Softmax-Policy Sampling

Require: Policy and factors Θ_z ; temperature coefficient β ; number of trajectories per sample U ; and number of time-steps per trajectory T .

```

function SOFTMAX_SAMPLING( $\Theta_z, U, T, \beta$ )
  for  $u = 1, \dots, U$  do
    Observe  $S_1^u$ 
    for  $t = 1, \dots, T$  do
       $[\mathbf{z}_{S_t^u}]_a \leftarrow \sum_{k=1}^K \prod_{d=1}^{D+1} [\Theta_d^z]_{i_d^{S_t^u}, A_t^u, k} \quad \forall a \in \mathcal{A}$ 
       $A_t^u \sim \mathcal{Z}(\mathbf{z}_{S_t^u}, \beta)$ 
      Take  $A_t^u$ , observe  $S_{t+1}^u$ , and  $R_t^u$ 
       $S_t^u \leftarrow S_{t+1}^u$ 
    end for
     $\tau_u = \{(S_t^u, A_t^u)\}_{t=1}^T$ 
  end for
  return  $\{\tau_u\}_{u=1}^U$ 
end function

```

following the softmax policy π_{Θ_z} for discrete actions. The sampling process for Gaussian policies is outlined in Algorithm 1, while the sampling mechanism for softmax policies is depicted in Algorithm 2.

Multi-dimensional action spaces. The framework presented in this paper can accommodate action spaces with multiple dimensions, denoted as $\mathcal{A} = \mathcal{A}_1 \times \dots \times \mathcal{A}_P$. Actions within this framework are represented as P -dimensional random vectors $a = [a_1, \dots, a_P]^\top$, sampled from a multivariate policy distribution. Traditionally, RL algorithms assume independence among action dimensions. For Gaussian policies this leads to a multivariate Gaussian distribution with an identity covariance matrix, which in the tensor framework can be modeled by introducing a new dimension in the tensor of means $\underline{\Theta}_\mu$,

Algorithm 3 Tensor Low-Rank Policy Gradient (TLR-PG)

Require: Initial policy and factors Θ^1 ; variable b that triggers Algorithm 1 if $b = 0$, and Algorithm 2 if $b = 1$; the parameters of the sampling function params_b ; learning rates $\{\eta^h\}_{h=1}^H$; number of trajectories per sample U ; maximum number of iterations H ; and number of time-steps per trajectory T .

```

1: for  $h = 1, \dots, H$  do
2:    $\{\tau_u^h\}_{u=1}^U \leftarrow \text{SAMPLING}(\Theta^h, U, T, b, \text{params}_b)$ 
3:    $\nabla_{\Theta} \bar{G}_V(\Theta^h) \leftarrow \text{Compute as per equation (14)}$ 
4:    $\Theta^{h+1} \leftarrow \Theta^h + \eta^h \nabla_{\Theta} \bar{G}_V(\Theta^h)$ 
5: end for

```

where $\mu_s = [\underline{\Theta}_\mu]_{i_s}$ represents a vector of means. Similarly, for policies based on the softmax distribution, introducing a set of logits per action dimension necessitates a new dimension in the tensor of logits $\underline{\Theta}_z$, where $\mathbf{Z}_s = [\underline{\Theta}_z]_{i_s}$ represents a matrix of logits. Due to the assumption of independent actions, the joint policy can be factorized as $\pi_{\Theta}(a|s) = \prod_{p=1}^P \pi_{\Theta}(a_p|s)$, simplifying the policy scores computation to $\nabla_{\Theta} \log \pi_{\Theta}(a|s) = \sum_{p=1}^P \nabla_{\Theta} \log \pi_{\Theta}(a_p|s)$. Hence, it is straightforward to extend the derivation of the policy scores presented in this section to the multi-dimensional case. For the sake of clarity, subsequent derivations in the paper focus on the unidimensional action space.

B. Tensor Low-Rank Actor-Critic Methods

Tensor low-rank policies can be readily integrated into PG and AC methods, since both are first-order methods. In this section, we introduce a *tensor low-rank* PG algorithm, and a *tensor low-rank* AC algorithm.

Policy Gradients. Recall that in PG we aim to maximize the function $\bar{G}_V$ defined in (1) via stochastic gradient ascent. Again, we use the Gaussian policy π_{Θ_μ} for continuous action spaces, and the softmax policy π_{Θ_z} for discrete action spaces.

The first step involves sampling a set of trajectories from the MDP $\{\tau_u\}_{u=1}^U \sim g(\Theta)$ using Algorithm 1 for the Gaussian case and Algorithm 2 for the softmax case. Then, we compute the sample returns $G_t^u = \sum_{t'=t}^T R_{t'}^u$, and the policy scores $\nabla_{\Theta} \log \pi_{\Theta}(A_t^u | S_t^u)$ using (12) for the Gaussian policy and (13) for the softmax policy. Finally, the factors in Θ are updated leveraging the stochastic approximation of the gradient defined in (2)

$$\tilde{\nabla}_{\Theta} \bar{G}_V(\Theta) = \frac{1}{U} \sum_{u=1}^U \sum_{t=1}^T G_t^u \nabla_{\Theta} \log \pi_{\Theta}(A_t^u | S_t^u). \quad (14)$$

The result is a tensor low-rank policy gradient (TLR-PG) algorithm that is described in Algorithm 3. The Gaussian case involves using the factors $\Theta = \Theta_\mu$, the learning rate η , and using the Gaussian sampling defined in Algorithm 1 with $b = 0$ and $\text{params}_b = \sigma$. Similarly, in the softmax case $\Theta = \Theta_z$, and we use the softmax sampling defined in Algorithm 2 with $b = 1$, and $\text{params}_b = \beta$.

Actor Critic. Tensor low-rank policies can be generalized to AC setups. The first steps are quite similar. Namely, following Algorithms 1 and 2, the actor π_{Θ_μ} or π_{Θ_z} is used to obtain a set of trajectories from the MDP $\{\tau_u\}_{u=1}^U$, and the samples are used to compute the policy scores according to (12) or (13). However, AC methods subtract the approximated VF from the sample return to estimate the advantage function $\mathbb{A}_\omega(S_t^u) = G_t^u - \mathbb{V}_\omega(S_t^u)$, resulting in algorithms with lower variance. The price to pay is that we need to learn the parameters ω of the VF approximator, or critic, minimizing the quadratic cost $L(\omega) = \frac{1}{2} \sum_{u=1}^U \sum_{t=1}^T (G_t^u - \mathbb{V}_\omega(S_t^u))^2$. Following the spirit of this paper, we propose a multilinear model to approximate the VFs as well. More specifically, we collect the VFs in the tensor $\underline{\mathbf{V}} \in \mathbb{R}^{S_1 \times \dots \times S_D}$ to then postulate a K' -rank PARAFAC model with factors $\omega = \{\mathbf{V}_d \in \mathbb{R}^{N_d \times K'}\}_{d=1}^D$. Then, the VF of s is $\mathbb{V}_\omega(s) = [\underline{\mathbf{V}}]_{i_s}$, which under the PARAFAC model are computed as $\sum_{k=1}^{K'} \prod_{d=1}^D [\mathbf{V}_d]_{i_d^s, k}$. In AC methods, the updates of the parameters of the actor and the critic are alternated. Firstly, we fix the critic and update the actor using the stochastic gradient

$$\tilde{\nabla}_\Theta \bar{G}_A(\Theta) = \frac{1}{U} \sum_{u=1}^U \sum_{t=1}^T \mathbb{A}_\omega(S_t^u) \nabla_\Theta \log \pi_\Theta(A_t^u | S_t^u), \quad (15)$$

where $\mathbb{A}_\omega(S_t^u) = G_t^u - [\underline{\mathbf{V}}]_{i_{S_t^u}}$. In the Gaussian case we set the factors $\Theta = \Theta_\mu$, while in the softmax case we set $\Theta = \Theta_z$. Secondly, we update the critic by minimizing $L(\omega)$ via gradient descent. In order to do so we need to compute the gradient $\nabla_\omega L(\omega)$, which entry-wise is

$$\frac{\partial L(\omega)}{\partial [\mathbf{V}_d]_{i_d, k}} = \sum_{u=1}^U \sum_{t=1}^T \left(G_t^u - [\underline{\mathbf{V}}]_{i_{S_t^u}} \right) \left(\prod_{j=1, j \neq d}^D [\mathbf{V}_j]_{i_j^{S_t^u}, k} \right) \mathcal{I}_{i_{i_d^{S_t^u}}}. \quad (16)$$

This leads to a tensor low-rank actor-critic (TLR-AC) algorithm which is fully depicted in Algorithm 4. In the Gaussian case $\Theta = \Theta_\mu$, and we use the sampling mechanism defined in Algorithm 1 with $b = 0$, and $\text{params}_b = \sigma$, while in the softmax case we set $\Theta = \Theta_z$ and the learning rate for the critic α , and we use the sampling in Algorithm 2 with $b = 1$, and $\text{params}_b = \beta$.

C. Tensor Low-Rank Trust-Region Methods

We claimed in previous sections that estimating the parameters of a policy via TR methods can be reduced to characterizing the policy scores too. In this section, we show first how TRPO and PPO can be expressed in terms of the policy scores, to then leverage our *tensor low-rank* policy models in both setups.

TRPO. We begin by recalling that in TRPO we aim to maximize $\bar{G}_T$ as defined in (5), subject to the constraint that the KL-divergence between π_Θ and the (old) sampling policy $\pi_{\tilde{\Theta}}$ is less than δ . More formally, we solve the optimization

$$\begin{aligned} \underset{\Theta}{\text{maximize}} \quad & \bar{G}_T(\Theta) = \mathbb{E}_{g(\tilde{\Theta})} \left[\sum_{t=1}^T q_{\Theta, \tilde{\Theta}}(S_t, A_t) \mathbb{A}_\omega(S_t) \right] \\ \text{subject to} \quad & \mathbb{E}[D_{KL}(\pi_{\tilde{\Theta}}(\cdot|s) || \pi_\Theta(\cdot|s))] \leq \delta \end{aligned} \quad (17)$$

Algorithm 4 Tensor Low-Rank Actor Critic (TLR-AC)

Require: Initial policy and factors Θ^1 ; initial VFs and factors ω^1 ; variable b that triggers Algorithm 1 if $b = 0$, and Algorithm 2 if $b = 1$; the parameters of the sampling function params_b ; actor learning rates $\{\eta^h\}_{h=1}^H$; critic learning rates $\{\alpha^h\}_{h=1}^H$; number of trajectories per sample U ; maximum number of iterations H ; and number of time-steps per trajectory T .

for $h = 1, \dots, H$ **do**

$\{\tau_u^h\}_{u=1}^U \leftarrow \text{SAMPLING}(\Theta^h, U, T, b, \text{params}_b)$

$\tilde{\nabla}_\Theta \bar{G}_A(\Theta^h) \leftarrow \text{Compute as per equation (15)}$

$\nabla_\omega L(\omega^h) \leftarrow \text{Compute as per equation (16)}$

$\Theta^{h+1} \leftarrow \Theta^h + \eta^h \tilde{\nabla}_\Theta \bar{G}_A(\Theta^h) \quad \triangleright \text{Actor update}$

$\omega^{h+1} \leftarrow \omega^h - \alpha^h \nabla_\omega L(\omega^h) \quad \triangleright \text{Critic update}$

end for

via sampling-based techniques. To render (17) tractable, we resort to Taylor expansions of both the objective function and the KL constraint [47], [49]. Leveraging a first-order approximation for the objective $\bar{G}_T$, and a second-order approximation for the KL constraint, the following quadratic optimization is reached:

$$\begin{aligned} \underset{\Theta}{\text{maximize}} \quad & \tilde{\mathbf{g}}^\top (\Theta - \tilde{\Theta}) \\ \text{subject to} \quad & (\Theta - \tilde{\Theta})^\top \tilde{\mathbf{H}} (\Theta - \tilde{\Theta}) \leq \delta, \end{aligned} \quad (18)$$

where $\tilde{\mathbf{g}} = \nabla_\Theta \bar{G}_T(\Theta) |_{\Theta=\tilde{\Theta}}$ is the gradient of the cost function w.r.t. the set of parameters Θ evaluated at $\tilde{\Theta}$, and $\tilde{\mathbf{H}}$ is the Hessian matrix of the KL constraint evaluated at $\tilde{\Theta}$, with entries $[\tilde{\mathbf{H}}]_{i,j} = \frac{\partial}{\partial \Theta_i} \frac{\partial}{\partial \Theta_j} \mathbb{E}[D_{KL}(\pi_{\tilde{\Theta}}(\cdot|s) || \pi_\Theta(\cdot|s))] |_{\Theta=\tilde{\Theta}}$. Notably, the gradient of the KL constraint evaluated at $\Theta = \tilde{\Theta}$ is zero and the Hessian evaluated at $\Theta = \tilde{\Theta}$ is the Fisher information matrix (FIM) [53]. The FIM definition allows us to reformulate $\tilde{\mathbf{H}}$ in terms of the outer product of the policy scores as $\tilde{\mathbf{H}} = \mathbb{E}[\nabla_\Theta \log \pi_\Theta(a|s) \nabla_\Theta \log \pi_\Theta(a|s)^\top] |_{\Theta=\tilde{\Theta}}$. Trivially, the gradient $\tilde{\mathbf{g}}$ can also be reformulated in terms of the policy scores $\nabla_\Theta \log \pi_\Theta$ as $\tilde{\mathbf{g}} = \mathbb{E}_{g(\tilde{\Theta})} \left[\sum_{t=1}^T q_{\Theta, \tilde{\Theta}}(S_t, A_t) \nabla_\Theta \log \pi_\Theta(A_t | S_t) \mathbb{A}_\omega(S_t) \right] |_{\Theta=\tilde{\Theta}}$. Recall that in TRPO we approximate the expectations in $\tilde{\mathbf{g}}$ and $\tilde{\mathbf{H}}$ using sampled trajectories from the MDP $\{\tau_u\}_{u=1}^U \sim g(\tilde{\Theta})$ following $\pi_{\tilde{\Theta}}$. Thus we end up having the following stochastic approximation

$$\tilde{\mathbf{g}} \approx \frac{1}{U} \sum_{u=1}^U \sum_{t=1}^T q_{\Theta, \tilde{\Theta}}(S_t^u, A_t^u) \nabla_\Theta \log \pi_\Theta(A_t^u | S_t^u) \mathbb{A}_\omega(S_t^u) \Big|_{\Theta=\tilde{\Theta}}, \quad (19)$$

$$\tilde{\mathbf{H}} \approx \frac{1}{U} \frac{1}{T} \sum_{u=1}^U \sum_{t=1}^T \nabla_\Theta \log \pi_\Theta(A_t^u | S_t^u) \nabla_\Theta \log \pi_\Theta(A_t^u | S_t^u)^\top \Big|_{\Theta=\tilde{\Theta}}. \quad (20)$$

As previously pointed out, equations (19) and (20) show that the policy model impacts the computation of $\tilde{\mathbf{g}}$ and $\tilde{\mathbf{H}}$ solely through the policy scores.

Algorithm 5 Tensor Low-Rank Trust-Region Policy Optimization (TLR-TRPO)

Require: Initial policy and factors Θ^1 ; initial VFs and factors ω^1 ; variable b that triggers Algorithm 1 if $b = 0$, and Algorithm 2 if $b = 1$; the parameters of the sampling function params_b ; critic learning rates $\{\alpha^h\}_{h=1}^H$; number of trajectories per sample U ; maximum number of iterations H ; and number of time-steps per trajectory T .

for $h = 1, \dots, H$ **do**

$\{\tau_u^h\}_{u=1}^U \leftarrow \text{SAMPLING}(\Theta^h, U, T, b, \text{params}_b)$

$\tilde{g} \leftarrow \text{Compute as per equation (19)}$

$\tilde{H} \leftarrow \text{Compute as per equation (20)}$

$\nabla_\omega L(\omega^h) \leftarrow \text{Compute as per equation (16)}$

$\Theta^{h+1} \leftarrow \text{Solve (18) using } \tilde{g} \text{ and } \tilde{H} \quad \triangleright \text{Actor update}$

$\omega^{h+1} \leftarrow \omega^h - \alpha^h \nabla_\omega L(\omega^h) \quad \triangleright \text{Critic update}$

end for

Now, everything is set up to introduce our *tensor low-rank* policy model in TRPO. First, we introduce the initial Gaussian policy $\pi_{\bar{\Theta}_\mu}$ for continuous action setups, and the initial softmax policy $\pi_{\bar{\Theta}_z}$ for discrete action spaces. Then, the actor step involves sampling a set of trajectories from the MDP $\{\tau_u\}_{u=1}^U$ following $\pi_{\bar{\Theta}_\mu}$ or $\pi_{\bar{\Theta}_z}$ as per Algorithms 1 and 2. We compute the sample returns $G_t^u = \sum_{t'=t}^T R_{t'}^u$, and the VFs using the critic $\nabla_\omega(S_t^u)$. Following (19) and (20), Gaussian policies lead to the stochastic gradient $\tilde{g}_\mu$ and the stochastic Hessian $\tilde{H}_\mu$ using the Gaussian policy scores in (12), and similarly, softmax policies lead to $\tilde{g}_z$ and $\tilde{H}_z$ using the softmax policy scores in (13). Lastly, we solve (18) via conjugate gradient descent methods [49] to obtain Θ_μ in the Gaussian case and Θ_z in the softmax case. After completing the actor step, the critic is updated in the exact same way as in Sec. III-B. The resulting tensor low-rank trust-region policy optimization (TLR-TRPO) algorithm is depicted in Algorithm 5. Again, the Gaussian case involves setting $\Theta = \Theta_\mu$, and sampling according to Algorithm 1 with $b = 0$ and $\text{params}_b = \sigma$. The softmax case involves setting $\Theta = \Theta_z$, and sampling according to Algorithm 2 with $b = 1$ and $\text{params}_b = \beta$.

PPO. In PPO we seek to maximize the unconstrained function $\bar{G}_P$ defined in (7), which enforces a trust-region via the clipped probability ratio $q_{\Theta, \bar{\Theta}}$. To solve this problem, we resort to first order methods, hence we need to compute the gradient $\nabla_\Theta \bar{G}_P(\Theta)$. Observing the definition of $q_{\Theta, \bar{\Theta}}$ in (6), the term inside the expectation of (7) can be rearranged as

$$\begin{aligned} & \min\{q_{\Theta, \bar{\Theta}}(S_t, A_t) \mathbb{A}_\omega(S_t), \hat{q}_{\Theta, \bar{\Theta}}(S_t, A_t) \mathbb{A}_\omega(S_t)\} \\ &= \begin{cases} (1 - \epsilon) \mathbb{A}_\omega(S_t) & \text{if } q_{\Theta, \bar{\Theta}}(S_t, A_t) \leq 1 - \epsilon \text{ and } \mathbb{A}_\omega(S_t) \leq 0 \\ (1 + \epsilon) \mathbb{A}_\omega(S_t) & \text{if } q_{\Theta, \bar{\Theta}}(S_t, A_t) \geq 1 + \epsilon \text{ and } \mathbb{A}_\omega(S_t) \geq 0 \\ q_{\Theta, \bar{\Theta}}(S_t, A_t) \mathbb{A}_\omega(S_t) & \text{otherwise.} \end{cases} \end{aligned} \quad (21)$$

Then, it is easy to see that, conditioned to the value of $\mathbb{A}_\omega(S_t)$, the gradient of $\bar{G}_P$ w.r.t. Θ outside the clipping range defined by

Algorithm 6 Tensor Low-Rank Proximal Policy Optimization (TLR-PPO)

Require: Initial policy and factors Θ^1 ; initial VFs and factors ω^1 ; variable b that triggers Algorithm 1 if $b = 0$, and Algorithm 2 if $b = 1$; the parameters of the sampling function params_b ; actor learning rates $\{\eta^h\}_{h=1}^H$; critic learning rates $\{\alpha^h\}_{h=1}^H$; number of trajectories per sample U ; maximum number of iterations H ; and number of time-steps per trajectory T .

for $h = 1, \dots, H$ **do**

$\{\tau_u^h\}_{u=1}^U \leftarrow \text{SAMPLING}(\Theta^h, U, T, b, \text{params}_b)$

$\tilde{\nabla}_\Theta \bar{G}_P(\Theta^h) \leftarrow \text{Compute as per equation (22)}$

$\nabla_\omega L(\omega^h) \leftarrow \text{Compute as per equation (16)}$

$\Theta^{h+1} \leftarrow \Theta^h + \eta^h \tilde{\nabla}_\Theta \bar{G}_P(\Theta^h) \quad \triangleright \text{Actor update}$

$\omega^{h+1} \leftarrow \omega^h - \alpha^h \nabla_\omega L(\omega^h) \quad \triangleright \text{Critic update}$

end for

ϵ is 0. Let $\mathcal{I}_{\Theta, \bar{\Theta}}^\omega(S_t, A_t)$ be an indicator function that takes the value 1 when the third clause of (21) is active, and 0 otherwise. The gradient of (7) is defined as

$$\nabla_\Theta \bar{G}_P(\Theta) = \mathbb{E}_{g(\bar{\Theta})} \left[\sum_{t=1}^T \mathcal{I}_{\Theta, \bar{\Theta}}^\omega(S_t, A_t) q_{\Theta, \bar{\Theta}}(S_t, A_t) \nabla_\Theta \log \pi_\Theta(A_t | S_t) \mathbb{A}_\omega(S_t) \right], \quad (22)$$

where again, computing the gradient of $\bar{G}_P$ boils down to estimating the policy scores $\nabla_\Theta \log \pi_\Theta(A_t | S_t)$.

Leveraging *tensor low-rank* policy models in PPO involves using the Gaussian policy $\pi_{\bar{\Theta}_\mu}$ or the softmax policy $\pi_{\bar{\Theta}_z}$ to first sample trajectories $\{\tau_u\}_{u=1}^U$ from the MDP as depicted in Algorithms 1 and 2. Next, we compute G_t^u , $\nabla_\omega(S_t^u)$, and the policy scores as per (12) or (13). We update the actor using the stochastic gradient in (22), and lastly, we update the critic. The resulting algorithm is called tensor low-rank proximal policy optimization (TLR-PPO), and is outlined in Algorithm 6. In the Gaussian case we set $\Theta = \Theta_\mu$, and we sample according to Algorithm 1 with $b = 0$ and $\text{params}_b = \sigma$. In the softmax case, we set $\Theta = \Theta_z$, and we sample as in Algorithm 2 with $b = 1$ and $\text{params}_b = \beta$.

D. Computational Complexity

The parsimonious tensor-based policy representation introduced in this paper entails a number of parameters $\sum_{d=1}^D N_d K$. Assuming, for simplicity, that the number of states per dimension is the same, $N_d = N_0$ for all d , the number of parameters reduces to $K D N_0$. This contrasts with the number of entries of the policy tensor $\Theta_\mu \in \mathbb{R}^{N_1 \times \dots \times N_D}$, which is $\prod_{d=1}^D N_d = N_0^D$. As a result, the number of parameters of our low-rank schemes grows linearly with the rank K , the number of dimensions D , and the number of (discrete) points per dimension N_0 . Note also that if one uses a finer grid and the number of points

per dimension increases to N'_0 , the number of parameters in our method increases by a factor of N'_0/N_0 , while the entries of the tensor increase as $(N'_0/N_0)^D$. The reduced number of parameters brings two main advantages: (a) fewer samples are required for convergence, as confirmed by our numerical experiments, which show that underparameterized tensor representations converge faster than NNs in practical setups; and (b) the proposed low-rank tensor models are usually computationally less expensive than NNs.

To further elaborate on advantage (b), consider the computation of the optimal action for the Gaussian case with a D -dimensional state space $\mathcal{S}$ and a single-dimensional action space¹ $\mathcal{A}$. In this case, we have the tensor of means $\underline{\Theta}_\mu \in \mathbb{R}^{N_1 \times \dots \times N_D}$, and the goal is to compute the mean associated with a given (current) state s . Given a state $s \in \mathcal{S}_1 \times \dots \times \mathcal{S}_D$, we do not need to recompute the entire tensor $\underline{\Theta}_\mu$ to obtain the corresponding mean action $[\underline{\Theta}_\mu]_s$. Instead, the computation can be carried out entry-wise by identifying the K -dimensional vectors associated with each of the D entries of state s and computing the mean via (8), which entails $K(D-1)$ multiplications and $K-1$ additions. Therefore, the complexity is on the order of $O(KD)$. Note that this cost does not depend on the cardinalities of the state dimensions $(N_1, \dots, N_D)$, but only on the dimensionality of the state space D and the rank K .

In the case of the NN, consider the multi-layer perceptron (MLP) for the same setup with an input vector associated with state s of dimensionality D , and a single output corresponding to the mean of the Gaussian distribution. Suppose the MLP has L layers, each with W hidden neurons. The computational complexity of such a model is on the order of $O(WD + (L-1)W^2)$. A few observations are in order. If the model has more than one layer, which is typically the case in deep learning, the complexity grows quadratically with W . On the other hand, if we consider a shallow network with $L=1$, the complexity reduces to $O(WD)$, which is comparable to that of the tensor model $O(KD)$, where the hidden size W and the rank K play analogous roles. However, single-layer models are usually shallow with $W \gg D$, whereas in the tensor case the rank K is on the order of D . Therefore, even in such situations, the tensor model can offer advantages from a computational complexity perspective.

Beyond inference, similar savings also appear in training. For each sampled state, our low-rank tensor-based schemes require updating only DK parameters, while an NN must update the full set of parameters. Taken together, these results show that tensor-based representations not only reduce sample requirements and inference costs, but also make training significantly more efficient. This combination of benefits underscores their practical value as scalable alternatives to NNs.

E. Convergence Analysis

We focus on the tensor low rank PG (TLR-PG) method in Algorithm 3 and analyze its convergence towards a stationary solution of (1). To make the analysis tractable, we consider a slight modification to the algorithm which composes of a

projection step on top of the stochastic policy gradient update. Let $\mathcal{O} \subset \mathbb{R}^{N_1 \times K} \times \dots \times \mathbb{R}^{N_D \times K}$ be a closed and convex set. In lieu of Line 4 of Algorithm 3, the update for Θ is

$$\Theta^{h+1} = \mathcal{P}_{\mathcal{O}}(\Theta + \eta^h \tilde{\nabla}_{\Theta} \bar{G}_V(\Theta^h)), \quad (23)$$

where $\mathcal{P}_{\mathcal{O}}(\cdot)$ denotes the Euclidean projection onto $\mathcal{O}$. For the sake of clarity, we define the shorthand notation $\nabla_{\Theta_i} \log p_{g(\Theta)}(\tau_T) := \sum_{t=1}^T \nabla_{\Theta_i} \log \pi_{\Theta}(A_t|S_t)$ and consider:

Assumption 1: The reward function is uniformly bounded, i.e., there exists a constant R such that

$$|r(s, a, s')| \leq R, \forall s, s' \in \mathcal{S}, a \in \mathcal{A}.$$

Assumption 2: For any $\Theta, \Theta' \in \mathcal{O}, s \in \mathcal{S}, a \in \mathcal{A}, i = 1, \dots, D$, there exists a constant L_0 such that

$$\|\nabla_{\Theta_i} \log \pi_{\Theta}(a|s) - \nabla_{\Theta_i} \log \pi_{\Theta'}(a|s)\| \leq L_0 \|\Theta - \Theta'\|.$$

Assumption 3: For any $\Theta, \Theta', \Theta'' \in \mathcal{O}, i = 1, \dots, D$, there exists a constant L_1 such that

$$\left\| \mathbb{E}_{\tau_T \sim g(\Theta)} [G_T(\tau_T) \nabla_{\Theta_i} \log p_{g(\Theta'')}(\tau_T)] - \mathbb{E}_{\tau'_T \sim g(\Theta')} [G_T(\tau'_T) \nabla_{\Theta_i} \log p_{g(\Theta'')}(\tau'_T)] \right\| \leq L_1 \|\Theta - \Theta'\|.$$

Assumption 4: For any Θ , there exists a constant G such that

$$\mathbb{E}_{\tau_T \sim g(\Theta)} \left[\|G_T(\tau_T) \nabla_{\Theta} \log p_{g(\Theta)}(\tau_T)\|^2 \right] \leq G^2.$$

Assumption 1 is a standard condition used in the RL literature. Meanwhile, Assumptions 2–4 can be verified for the Gaussian and softmax policies if Θ is compact; see Appendix A. The Lipschitz constants introduced in Assumptions 2 and 3 reflect the influence of the tensor parameterization in the convergence analysis, as they scale as $O(K \sum_{i=1}^D N_i)$. Consequently, lower-rank and lower-dimensional models lead to smaller Lipschitz constants and, therefore, smoother behavior. To handle the projected stochastic PG algorithm (23) for non-convex optimization problems, our analysis relies on the framework in [54] that utilizes the following Moreau envelope of the objective function in (1): for any $\lambda > 0$, we set

$$\bar{G}_V^\lambda(\Theta) := \min_{\vartheta \in \mathcal{O}} \left\{ -\bar{G}_V(\vartheta) + \frac{1}{2\lambda} \|\Theta - \vartheta\|_F^2 \right\}.$$

For sufficiently small λ , it holds for any Θ [54],

$$\left\| \frac{1}{\lambda} (\Theta - \mathcal{P}_{\mathcal{O}}(\Theta + \lambda \nabla \bar{G}_V(\Theta))) \right\|^2 \leq 3 \|\nabla \bar{G}_V^\lambda(\Theta)\|^2, \quad (24)$$

which leads to the following convergence guarantee for the projected TLR-PG method.

Theorem 1: Under Assumptions 1–4. For any $\bar{L} > L_G := D(T^2 L_0 R + L_1)$ and maximum iteration number $H \geq 1$, the following holds for $\{\Theta^h\}_{h=1}^H$ generated by (23):

$$\begin{aligned} & (1/H) \sum_{h=1}^H \mathbb{E} \left[\|\nabla_{\Theta} \bar{G}_V^{1/\bar{L}}(\Theta^h)\|^2 \right] \\ & \leq \frac{\bar{L}}{\bar{L} - L_G} \frac{\bar{G}_V^{1/\bar{L}}(\Theta^1) - \bar{G}_V^{1/\bar{L},*} + \frac{\bar{L}G^2}{2} \sum_{h=1}^H (\eta^h)^2}{\sum_{h=1}^H \eta^h}, \end{aligned} \quad (25)$$

where $\bar{G}_V^{1/\bar{L},*} := \min_{\Theta \in \mathcal{O}} \bar{G}_V^{1/\bar{L}}(\Theta)$.

Proof: The proof follows directly from [54, Theorem 3.1]. In particular, it suffices to show that $\nabla \bar{G}_V(\Theta)$ is L_G -Lipschitz

¹The same analysis extends to softmax policies and multi-dimensional action spaces, but is omitted here for the sake of readability.

continuous, which implies that $\bar{G}_V(\Theta)$ is L_G -weakly convex. We observe that for any $i = 1, \dots, D$,

$$\begin{aligned} & \|\nabla_{\Theta_i} \bar{G}_V(\Theta) - \nabla_{\Theta_i} \bar{G}_V(\Theta')\| \\ & \leq \|\mathbb{E}_{\tau_T \sim g(\Theta)} [G_T(\tau_T)(\nabla_{\Theta_i} \log p_{g(\Theta)}(\tau_T) - \log p_{g(\Theta')}(\tau_T))]\| \\ & \quad + \|\mathbb{E}_{\tau_T \sim g(\Theta')} [G_T(\tau_T) \nabla_{\Theta_i} \log p_{g(\Theta')}(\tau_T)] \\ & \quad - \mathbb{E}_{\tau_T \sim g(\Theta)} [G_T(\tau_T) \nabla_{\Theta_i} \log p_{g(\Theta')}(\tau_T)]\|. \end{aligned}$$

We note that the last term is bounded by $L_1 \|\Theta - \Theta'\|$ under Assumption 3, while the first term is bounded by $T^2 L_0 R \|\Theta - \Theta'\|$ under Assumptions 1 and 2. Summing up over $i = 1, \dots, D$ and rearranging terms shows that $\|\nabla_{\Theta} \bar{G}_V(\Theta) - \nabla_{\Theta} \bar{G}_V(\Theta')\| \leq L_G \|\Theta - \Theta'\|$.

Set $\eta^h = 1/\sqrt{H}$. Together with (24), for sufficiently large H , the above shows that in H iterations, the projected TLR-PG method finds an $O(1/\sqrt{H})$ -stationary solution to (1). $\square$

Theorem 1 establishes that the projected TLR-PG method converges sublinearly to a neighborhood of a stationary solution, with size inversely proportional to $\sqrt{H}$ and proportional to $\bar{L}$. This quantifies the effect of the tensor parametrization on convergence. In particular, for sufficiently large $H \gg 1$ and $\eta^h = 1/\sqrt{H}$, the convergence neighborhood is bounded by

$$\frac{\bar{L}}{\bar{L} - L_G} \frac{\bar{G}_V^{1/\bar{L}}(\Theta^1) - \bar{G}_V^{1/\bar{L},*}}{\sqrt{H}}.$$

By setting $\bar{L} = 2L_G$ and noting that $\bar{G}_V^{1/\bar{L}}(\Theta^1) - \bar{G}_V^{1/\bar{L},*} = O(\bar{L})$, this simplifies to

$$O\left(\frac{D(T^2 L_0 R + L_1)}{\sqrt{H}}\right),$$

where we recall that the Lipschitz constants L_0 and L_1 are in the order $O(K \sum_{i=1}^D N_i)$. In a nutshell, the smaller the tensor, that is, the fewer the dimensions D and the lower the rank K , the smaller the convergence neighborhood for a fixed number of iterations H .

Under the general tensor low-rank model considered in this paper, convergence to the global optimum cannot be guaranteed, as the model is multilinear (non-convex) rather than tabular (convex) [36], [38], and no strong assumptions such as the PL condition are imposed [35], [39]. Regarding convergence to local optima, our results improve upon existing work [40] in that: (i) by leveraging the tensor model, we can verify smoothness for tensor-based Gaussian and softmax policies (see Appendix A); (ii) we guarantee unbiased gradients by considering fixed-length trajectories; and (iii) we characterize the size of the convergence neighborhood through the tensor model via $\bar{L}$, showing that it shrinks as the rank of the tensor of parameters decreases.

IV. NUMERICAL EXPERIMENTS

The aim of this section is to illustrate empirically that low-rank tensor policies can attain similar returns to NN-based methods while demanding fewer resources. More specifically, we show that low-rank tensor policies require

less amount of parameters and time to converge. We assess this claim by: i) thoroughly investigating diverse policy-based algorithmic configurations across various classical control problems, followed by ii) an examination of a (more realistic) higher-dimensional wireless communications setup. The code, along with a more detailed description of the experiments, is available in the GitHub repository linked to this paper [55].

A. Classical Control Problems

In order to evaluate the performance of the proposed methods we have selected two well-known control problems implemented in the toolkit OpenAI Gym [56]:

- *MountainCar*, where a car tries to reach the top of a hill. The state-action space has 3 dimensions, and the agent attains a positive reward upon reaching the goal, but incurs a penalty proportional to the fuel expended.
- *Pendulum*, where the goal is to keep a pendulum pointing upwards. The state-action space has 4 dimensions, and the reward function weights the position of the pendulum and the torque developed to control it.

The described environments admit a continuous and a discrete action-space version.

Empirical low-rankness. Firstly, we have verified our assertion regarding the low-rank nature of parameter tensors empirically.

To this end, we trained a NN using PPO on both discrete and continuous versions of the Pendulum and MountainCar problems. In addition, we considered two other classical environments: the CartPole problem, where a pendulum is balanced on a horizontally moving cart, and the Goddard Rocket problem [57], where the objective is to maximize altitude for a limited fuel budget. Subsequently, we have discretized the state space and employed the learned policies to evaluate the parameters across all discretized states.

These parameters were then collected into a tensor of Gaussian means or softmax logits. For instance, in the case of the Pendulum, the tensor of Gaussian means has $100 \times 100 \times 100$ entries, and the tensor of softmax logits has $100 \times 100 \times 100 \times 3$ entries. The maximum possible rank of the former is $K_{\max} = 10,000$, while for the latter is $K'_{\max} = 30,000$. Following this, we have decomposed the tensors using the PARAFAC decomposition for various ranks. To evaluate the approximation performance, we have computed the Normalized Frobenius Error (NFE) and the Normalized Maximum Error (NME) between a given tensor $\underline{\mathbf{X}}$ and its approximation $\tilde{\underline{\mathbf{X}}}$, which are defined as

$$\begin{aligned} \text{NFE} &= \|\underline{\mathbf{X}} - \tilde{\underline{\mathbf{X}}}\|_F / \|\underline{\mathbf{X}}\|_F, \\ \text{NME} &= \|\underline{\mathbf{X}} - \tilde{\underline{\mathbf{X}}}\|_{\infty} / \|\underline{\mathbf{X}}\|_{\infty}, \end{aligned}$$

where, with a slight abuse of notation, the infinity norm is taken over the vectorized form of the corresponding tensors. The results are depicted in Fig. 1. As observed, NFE and NME decreases sharply as the rank increases for all problems. This indicates that of most of the energy is concentrated in the first factors. Consequently, relatively low-rank approximations of the parameter tensors closely match the original discretized values estimated by NN methods in terms of Frobenius and infinity

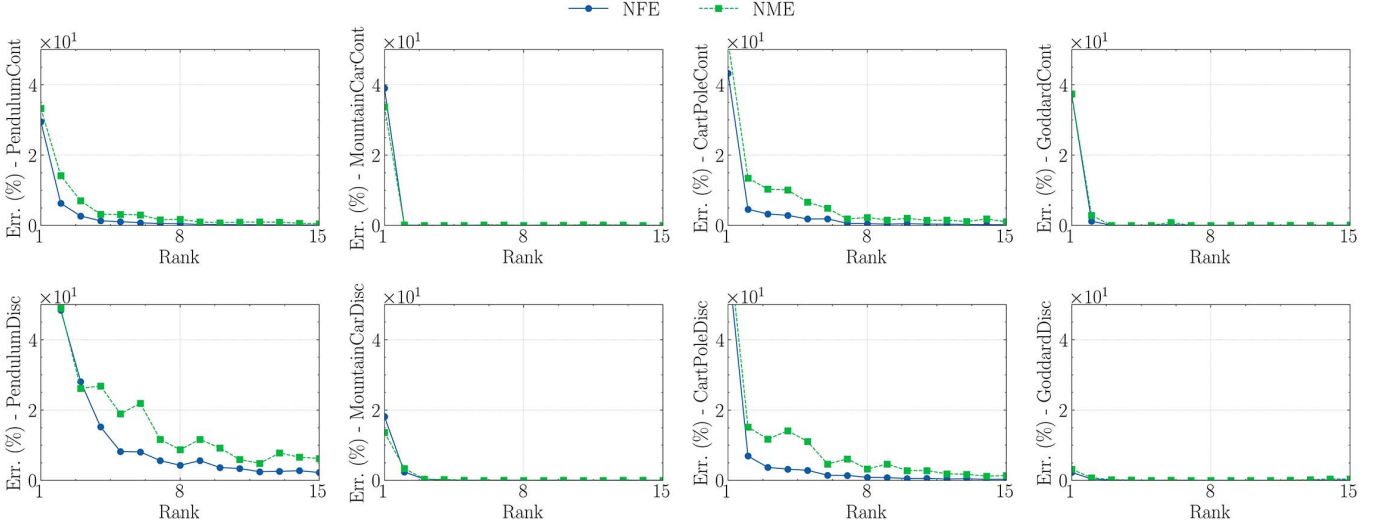


Fig. 1. NFE and NME between the tensor of Gaussian and softmax parameters obtained via NN-based PPO and their low-rank PARAFAC decomposition in the continuous and discrete variants of the Pendulum, MountainCar, CartPole and Goddard environments. The errors decrease as the rank of the approximation increases.

norms. Remarkably, despite the potentially high maximum rank of both tensors, the Gaussian mean tensors show a small NFE with rank set to 5 in the Pendulum and CartPole environments, while the softmax-logit tensors achieve a comparable error at slightly higher ranks. In these setups, the NME, which captures the largest approximation errors, is slightly larger and noisier, but consistently decays at the same rate as the NFE. For the MountainCar and Goddard Rocket problems, even with small ranks such as $K = 3$, both errors decay nearly to zero in the Gaussian and softmax cases. Unlike the Pendulum and CartPole environments, the defining feature of MountainCar and Goddard Rocket is that the reward is sparse, suggesting a potential connection between sparsity of rewards and the low-rankness of the resulting policy representations. In plain words, NN methods learn low-rank representations of the policies, even without explicitly imposing it. This underscores the potential of low rank for designing parsimonious policies. Interestingly, this finding is in agreement with the observations in [18], which reports that the VFs of many classical control problems, when arranged in a tensor form, exhibit a low-rank.

Continuous action spaces. We have used Gaussian policies for dealing with continuous action spaces. We have compared the proposed tensor low-rank policies against their NN-based counterparts. We have also considered a baseline linear policy model with radial basis function (RBF) features [58]. Specifically, we have compared TLR-AC, as described in Algorithm 4, with a linear-based AC algorithm (RBF-AC), and a NN-based AC algorithm (NN-AC); TLR-TRPO, as described in Algorithm 5, against a linear-based TRPO algorithm (RBF-TRPO), and a NN-based TRPO algorithm (NN-TRPO); and TLR-PPO, as described in Algorithm 6, against a linear-based PPO algorithm (RBF-PPO), and a NN-based PPO algorithm (NN-PPO). To assess the convergence speed of the proposed algorithms we have measured the return, or cumulative reward, per episode. Convergence is then quantified by how rapidly the algorithm

achieves high returns. We have tuned the rank K of our tensor low-rank policies and the hyper-parameters of the NNs via exhaustive search to identify the most compact models, in terms of parameters, capable of yielding high returns. For fairness in comparison, we have set σ , which controls the exploration in Gaussian policies, and the hyper-parameters that control the computational budget, such as the number of trajectories for the update U , to the same values across both tensor low-rank and NN policies. We have conducted 100 experiments of each combination of setup-algorithm, and the median returns are depicted in Fig. 2.

While linear methods might involve a similar number of parameters in the PendulumCont scenario, tensor low-rank methods exhibit significantly faster convergence in both setups. Moreover, tensor low-rank policies exhibit superior performance in terms of achieved returns. Tensor-based methods demand fewer parameters than their NN counterparts across all scenarios. This parameter efficiency translates into superior convergence speed for our proposed algorithms. Remarkably, despite the efficiency gains, we do not significantly compromise on return, with only one exception: NN-TRPO achieves marginally higher returns than TLR-TRPO in the PendulumCont. Moreover, our analysis reveals another advantage of tensor low-rank models. The variance in return achieved by TLR-AC is notably smaller than that of NN-AC in the PendulumCont environment.

Our numerical experiments further corroborate the theoretical insights introduced in Section III-E. In the MountainCar problem, both the dimensionality and the rank of the optimal policy are smaller than in the Pendulum case (see Fig. 1). This leads to smaller Lipschitz constants L_0 and L_1 , as discussed in Section III-E. Consequently, Theorem 1 predicts a smaller convergence neighborhood for MountainCar, indicating that the algorithm stabilizes around solutions of smaller variance. Empirically, this manifests as trajectories that reach their steady

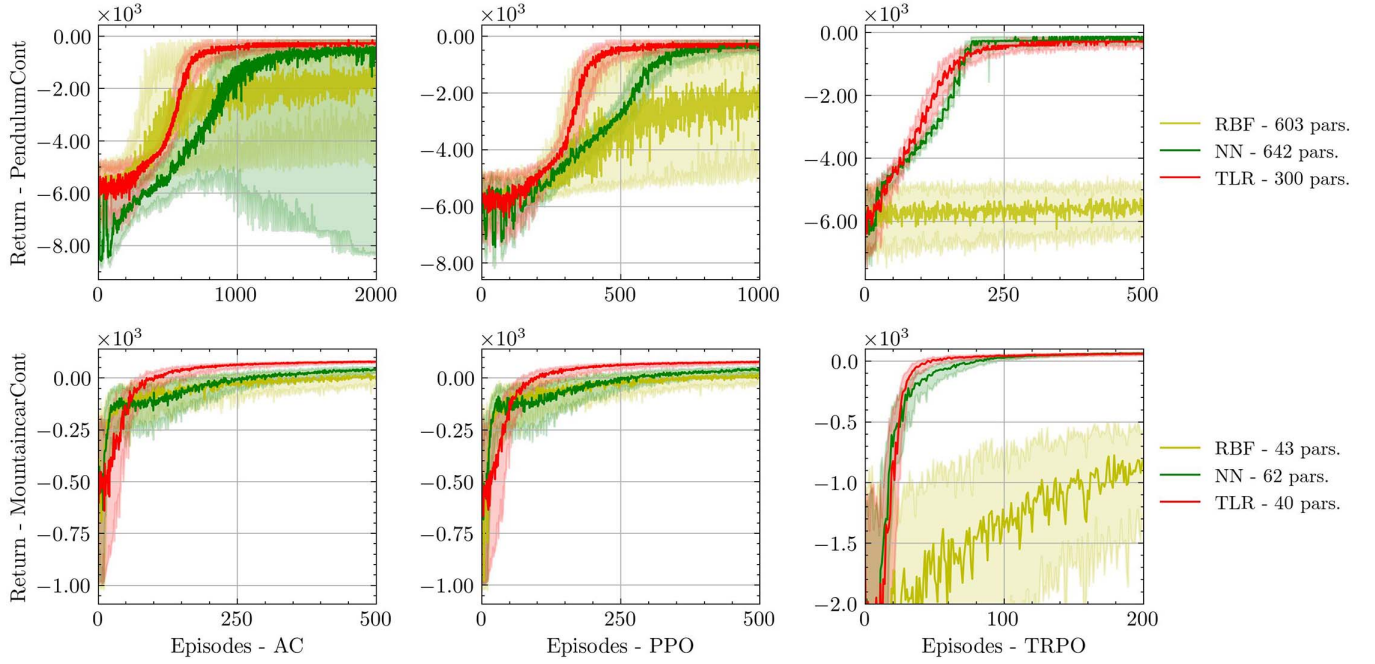


Fig. 2. Median return per episode of tensor low-rank policies (TLR) against NN-based policies (NN) across 100 experiments in various continuous action-space RL problems and algorithmic setups. The displayed confidence interval are the interquartile ranges. TLR policies converge faster than NN-based policies in almost all the scenarios. Furthermore, they require less parameters.

region earlier and fluctuate less around the limit point compared to those of the Pendulum environment.

Discrete action spaces. To deal with the discrete action-space variants of the previously introduced environments, we have used softmax policies. Again, we have compared TLR-AC against RBF-AC and NN-AC, TLR-TRPO against RBF-TRPO and NN-TRPO, and TLR-PPO against RBF-PPO and NN-PPO. We have used the return per episode as the main metric, and tuned the hyper-parameters via exhaustive search. Also, we have set the parameter β and the hyper-parameters that control the computational budget to the same values, and we have conducted 100 experiments of each combination of setup-algorithm. The median returns are shown in Fig. 3.

The results are consistent with the previous experiment. Tensor low-rank policies routinely outperform linear methods in terms of convergence speed and achieved return even with the linear method requiring less parameters in the PendulumDisc scenario. Furthermore, they not only demand fewer parameters than NN-based algorithms but converge faster as well, except for the PendulumDisc environment, where NN-TRPO demonstrates faster convergence than TLR-TRPO. Regarding the return, the only case where NN-PPO achieves marginally higher returns than TLR-PPO is the PendulumDisc environment. Again, we observe that the variance in the return achieved by TLR-AC is similar to that of NN-AC, except for the MountainCarDisc environment, where our method exhibits a clearly smaller variance. These findings highlight the potential of multilinear tensor low-rank policy models. They strike a balance by being parameter-efficient and converging quickly, while achieving returns comparable to those of NNs. This underscores the relevance of low-rank models in RL scenarios.

B. Wireless Communications Setup

This section examines a wireless communications setup with opportunistic multiple-access. This is not only a more realistic setup, but also a larger problem that helps us to assess how well our methods scale. Specifically, a single agent dispatches packets to an access point using two orthogonal channels ($C = 2$). The packets, which arrive every $T = 10$ time-slots, accumulate in a queue. Access is opportunistic, and the channels may often be occupied. Sending packets consumes energy from a battery, that recharges over time through energy harvesting. The corresponding MDP encompasses a state space of $D_S = 6$ dimensions: the fading intensity and occupancy status across the two channels, battery energy levels, and queue packet counts. Each time-step, the agent determines the power allocation for the duo of channels, thus defining an action space of $D_A = 2$ continuous dimensions. Transmission rates follow Shannon's capacity formula, and we establish that 80 % of the packets are lost when channels are occupied. The reward function accounts for the battery level (positively weighted) and the queue size (negatively weighted), compelling the agent to trade-off transmission throughput and battery levels. More details can be found in [55].

Given the continuous and multi-dimensional nature of the action space, we have used a multivariate Gaussian policy. We focused on comparing TLR-PPO against NN-PPO, due to the widespread utilization of PPO as a policy-based algorithm. Again, we have used returns per episode to evaluate the proposed algorithms, which were tuned for compactness through exhaustive search. Median returns are depicted in Fig. 4 based on 100 experiments. In essence, TLR-PPO not only requires

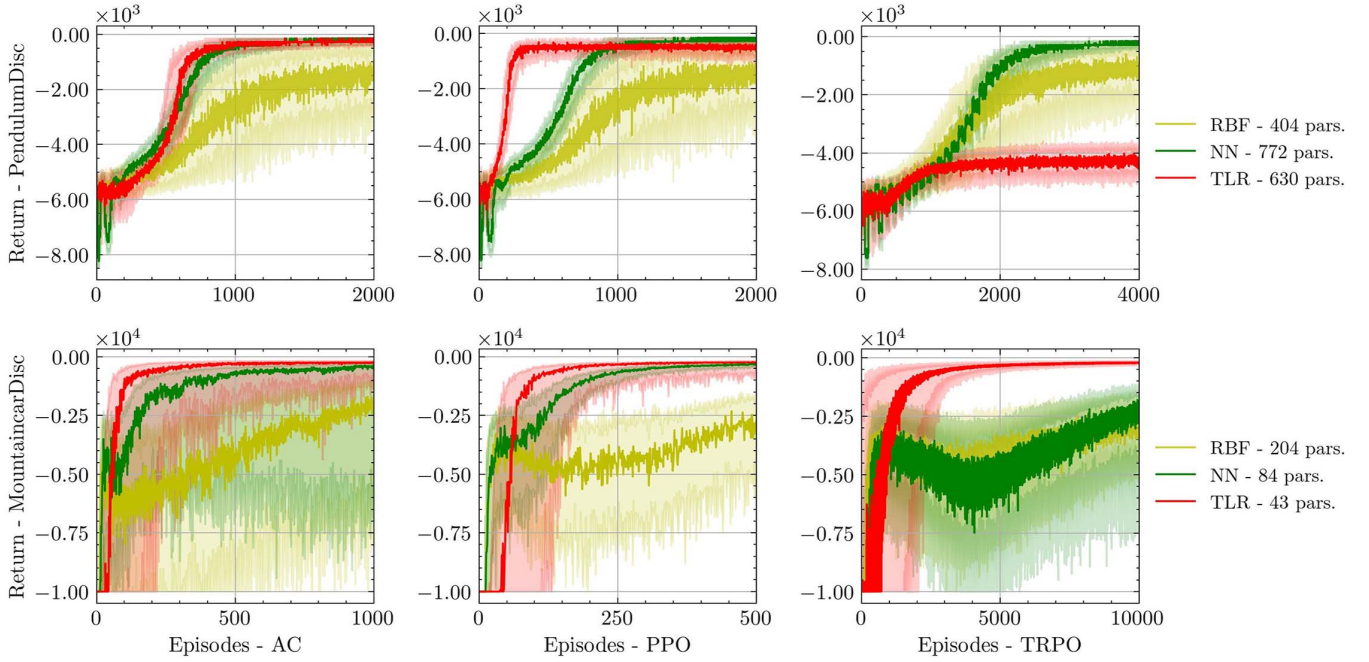


Fig. 3. Median return per episode of tensor low-rank policies (TLR) against NN-based policies (NN) across 100 experiments in various discrete action-space RL problems and algorithmic setups. The displayed confidence interval are the interquartile ranges. TLR policies converge faster than NN-based policies in almost all the scenarios. Furthermore, they require less parameters.

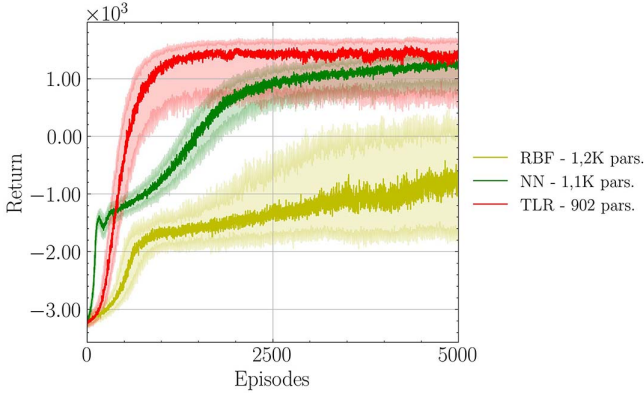


Fig. 4. Median return per episode of tensor low-rank policies (TLR) against NN-based policies (NN) across 100 experiments in the wireless communications setup. The displayed confidence interval are the interquartile ranges. TLR-PPO converges faster than NN-PPO while requiring less parameters.

fewer parameters than NN-PPO but also demonstrates superior empirical convergence speed. Remarkably, despite the modest difference in parameter count between TLR-PPO and NN-PPO, the former achieves steady-state much more rapidly. This shows that by capitalizing on the structure of the MDP via low-rank methods, TLR-PPO enables faster and more stable training, thereby reducing the computational resources needed for learning.

V. CONCLUSION

This paper introduced multilinear tensor *low-rank* policy models in the context of policy-based RL. Specifically,

it harnessed tensor-completion techniques to approximate Gaussian and softmax policies via multilinear mappings in policy-gradient RL setups. Within this framework, four tensor *low-rank* algorithms have been proposed: tensor low-rank policy gradient (TLR-PG), tensor low-rank actor-critic (TLR-AC), tensor low-rank trust-region policy optimization (TLR-TRPO), and tensor low-rank proximal policy optimization (TLR-PPO). The proposed algorithms are i) easy to adjust since they involve tuning one single model hyper-parameter, the rank of the tensor; ii) convergent under mild conditions as demonstrated theoretically; and iii) potentially faster than their NN-based counterparts as showed through numerical experiments in several standard benchmarks. By capitalizing on the inherent low-rank structure of several RL setups, these approaches aim to mitigate issues commonly encountered in NN-based methods. In a nutshell, building on the fact that low rank is prevalent in RL, this paper put forth a systematic approach to incorporating low-rank multilinear structure in policy-based RL. Beyond its immediate impact, this work opens several research directions in tensor low-rank methods for policy-based RL, including: (i) studying the identifiability of policy representations under mild conditions, (ii) exploring the interplay between low-rank policy parameterizations and low-rank value functions or MDPs, and (iii) designing neural architectures that embed tensor low-rank priors, thereby combining the expressivity and ability to handle continuous spaces with the advantages of the parsimonious schemes proposed in this paper.

APPENDIX A VERIFYING THE ASSUMPTIONS

Notice that under Assumption 1, $\sup_{\tau_T} |G_T(\tau_T)| \leq RT$ and we recall that

$$\begin{aligned} \nabla_{\Theta} \bar{G}_V(\Theta) &= \mathbb{E}_{g(\Theta)} \left[G_T(\tau_T) \sum_{t=1}^T \nabla_{\Theta} \log \pi_{\Theta}(A_t | S_t) \right] \\ &= \int G_T(\tau_T) \nabla \log p_{g(\Theta)}(\tau_T) p_{g(\Theta)}(\tau_T) d\tau_T. \end{aligned} \quad (26)$$

As such, it suffices to prove that $\nabla_{\Theta} \log \pi_{\Theta}(A_t^u | S_t^u)$ is Lipschitz and bounded to verify the required assumptions in Sec. III-E. Hereafter, we assume that Θ is bounded such that for any $\Theta = \{\Theta_d\}_{d=1}^D \in \mathcal{O}$, $\max_{d=1, \dots, D} \|\Theta_d\|_{\infty} \leq B$. Note that this implies $\|[\Theta]_{i_s}\| \leq KB^D$. We observe the following cases.

For the *Gaussian policy*, we recall that

$$\frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\Theta_d]_{i,k}} = \frac{a - [\Theta]_{i_s}}{\sigma^2} \left(\prod_{j=1 \neq d}^D [\Theta_j]_{i_d^s, k} \right) \mathcal{I}_{i=i_d^s}.$$

Assume that the actions are always bounded with $|a| \leq A$ for any $a \in \mathcal{A}$, we can verify Assumption 4 as

$$\left| \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\Theta_d]_{i,k}} \right| \leq \frac{A + KB^D}{\sigma^2} B^{D-1}, \quad (27)$$

which leads to $G^2 < \infty$. Also, Assumption 2 can be verified by noting that for any Θ, Θ'

$$\begin{aligned} &\left| \frac{\partial \log \pi_{\Theta}(a|s)}{\partial [\Theta_d]_{i,k}} - \frac{\partial \log \pi_{\Theta'}(a|s)}{\partial [\Theta_d]_{i,k}} \right| \\ &\leq \mathcal{I}_{i=i_d^s} \left(\frac{B^{2D-2}}{\sigma^2} + \frac{(A + KB^D)B^{D-2}}{\sigma^2} \right) \\ &\quad \sum_{d=1}^D |[\Theta_d]_{i_d^s, k} - [\Theta'_d]_{i_d^s, k}| \end{aligned} \quad (28)$$

This leads to $L_0 < \infty$. Finally, to verify Assumption 3, we have

$$\begin{aligned} |p_{g(\Theta)}(\tau_T) - p_{g(\Theta')}(\tau_T)| &\leq \frac{1}{4} \sum_{t=1}^T |\mu_{\Theta}(s_t) - \mu_{\Theta'}(s_t)| \\ &\leq \frac{DTB^{D-1}}{4} \|\Theta - \Theta'\| \end{aligned} \quad (29)$$

where we used the fact that the Gaussian distribution function is $1/4$ -Lipschitz. Combining with (26) leads to $L_1 < \infty$.

For the *softmax policy*, we similarly recall that

$$\begin{aligned} \frac{\partial \log \pi_{\Theta_z}(a|s)}{\partial [\Theta_z^z]_{i,k}} &= \sum_{b=1}^C (\mathcal{I}_{a=b} - \mathcal{Z}([\Theta_z]_{i_s}, \beta)_b) \\ &\quad \left(\prod_{j=1 \neq d}^{D+1} [\Theta_j^z]_{i_j^s, b, k} \right) \mathcal{I}_{i=i_d^s, b}, \end{aligned}$$

Assume that $|\mathcal{Z}(\cdot)_a| \leq Z$ for all $a \in \mathcal{A}$, and recalling $\Theta \in \mathcal{O}$ imply

$$\left| \frac{\partial \log \pi_{\Theta_z}(a|s)}{\partial [\Theta_z^z]_{i,k}} \right| \leq CB^D Z < \infty$$

which further leads to Assumption 4. Assumption 2 can be verified following the same steps as in the Gaussian policy case

[cf. (28)]. Assumption 3 can be verified using the fact that the soft-max function is 1-Lipschitz.

ACKNOWLEDGMENT

Minor edits of this document were made with the assistance of ChatGPT.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [2] D. P. Bertsekas, *Reinforcement Learning and Optimal Control*. Belmont, MA, USA: Athena Scientific, 2019.
- [3] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [4] D. Silver et al., "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [5] T. Brown et al., "Language models are few-shot learners," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, vol. 33, pp. 1877–1901.
- [6] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, vol. 1. Belmont, MA, USA: Athena Scientific, 2000.
- [7] D. P. Bertsekas and J. N. Tsitsiklis, *Neuro-Dynamic Programming*. Belmont, MA, USA: Athena Scientific, 1996.
- [8] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 1999, vol. 12, pp. 1057–1063.
- [9] S. Paternain, J. A. Bazerque, A. Small, and A. Ribeiro, "Stochastic policy gradient ascent in reproducing kernel Hilbert spaces," *IEEE Trans. Autom. Control*, vol. 66, no. 8, pp. 3429–3444, Aug. 2021.
- [10] T. Chen, K. Zhang, G. B. Giannakis, and T. Başar, "Communication-efficient policy gradient methods for distributed reinforcement learning," *IEEE Trans. Control Netw. Syst.*, vol. 9, no. 2, pp. 917–929, Jun. 2022.
- [11] Y. Luo, C.-C. Chiu, N. Jaitly, and I. Sutskever, "Learning online alignments with continuous rewards policy gradient," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2017, pp. 2801–2805.
- [12] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [13] G. Lever, J. Shawe-Taylor, R. Stafford, and C. Szepesvári, "Compressed conditional mean embeddings for model-based reinforcement learning," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2016, vol. 30, pp. 1779–1787.
- [14] E. Tolstaya, A. Koppel, E. Stump, and A. Ribeiro, "Nonparametric stochastic compositional gradient descent for Q-learning in continuous Markov decision problems," in *Proc. Ann. Amer. Control Conf. (ACC)*, 2018, pp. 6608–6615.
- [15] Q. Hu, S. Shi, Y. Cai, and G. Yu, "DDPG-driven deep-unfolding with adaptive depth for channel estimation with sparse Bayesian learning," *IEEE Trans. Signal Process.*, vol. 70, pp. 4665–4680, 2022.
- [16] Y. Yang, G. Zhang, Z. Xu, and D. Katabi, "Harnessing structures for value-based planning and reinforcement learning," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2019.
- [17] K.-C. Tsai et al., "Tensor-based reinforcement learning for network routing," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 3, pp. 617–629, Apr. 2021.
- [18] S. Rozada, S. Paternain, and A. G. Marques, "Tensor and matrix low-rank value-function approximation in reinforcement learning," *IEEE Trans. Signal Process.*, vol. 72, pp. 1634–1649, 2024.
- [19] S. Rozada and A. G. Marques, "Matrix low-rank approximation for policy gradient methods," in *IEEE Intl. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.
- [20] C. Eckart and G. Young, "The approximation of one matrix by another of lower rank," *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936.
- [21] I. Markovsky, *Low Rank Approximation*. New York, NY, USA: Springer-Verlag, 2012.
- [22] M. Udell et al., "Generalized low rank models," *Found. Trends Mach. Learn.*, vol. 9, no. 1, pp. 1–118, 2016.
- [23] T. G. Kolda and B. W. Bader, "Tensor decompositions and applications," *SIAM Rev.*, vol. 51, no. 3, pp. 455–500, 2009.
- [24] N. D. Sidiropoulos, L. De Lathauwer, X. Fu, K. Huang, E. E. Papalexakis, and C. Faloutsos, "Tensor decomposition for signal processing and machine learning," *IEEE Trans. Signal Process.*, vol. 65, no. 13, pp. 3551–3582, Jul. 2017.
- [25] A. Agarwal, S. Kakade, A. Krishnamurthy, and W. Sun, "Flambe: Structural complexity and representation learning of low rank MDPs," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 20095–20107.

- [26] M. Uehara, X. Zhang, and W. Sun, "Representation learning for online and offline RL in low-rank MDPs," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2021.
- [27] A. M. Barreto, R. L. Beirigo, J. Pineau, and D. Precup, "Incremental stochastic factorization for online reinforcement learning," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2016, vol. 30, pp. 1468–1475.
- [28] N. Jiang, A. Krishnamurthy, A. Agarwal, J. Langford, and R. E. Schapire, "Contextual decision processes with low Bellman rank are PAC-learnable," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, vol. 70, pp. 1704–1713.
- [29] K. Azizzadenesheli, A. Lazaric, and A. Anandkumar, "Reinforcement learning of POMDPs using spectral methods," in *Proc. Conf. Learn. Theory (CLT)*, 2016, pp. 193–256.
- [30] A. Mahajan et al., "Tesseract: Tensorised actors for multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 7301–7312.
- [31] T. Sam, Y. Chen, and C. L. Yu, "Overcoming the long horizon barrier for sample-efficient reinforcement learning with latent low-rank structure," *ACM Meas. Anal. Comput. Syst.*, vol. 7, no. 2, pp. 1–60, 2023.
- [32] D. Shah, D. Song, Z. Xu, and Y. Yang, "Sample efficient reinforcement learning via low-rank matrix estimation," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, vol. 33, pp. 12092–12103.
- [33] S. Rozada, V. Tenorio, and A. G. Marques, "Low-rank state-action value-function approximation," in *Proc. Eur. Signal Process. Conf. (EUSIPCO)*, 2021, pp. 1471–1475.
- [34] S. Rozada and A. G. Marques, "Matrix low-rank trust region policy optimization," in *Proc. IEEE Int. Workshop Computat. Adv. Multi-Sensor Adaptive Process. (CAMSAP)*, 2023, pp. 1–5.
- [35] J. Mei, C. Xiao, C. Szepesvari, and D. Schuurmans, "On the global convergence rates of softmax policy gradient methods," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 6820–6829.
- [36] A. Agarwal, S. M. Kakade, J. D. Lee, and G. Mahajan, "On the theory of policy gradient methods: Optimality, approximation, and distribution shift," *J. Mach. Learn. Res.*, vol. 22, no. 98, pp. 1–76, 2021.
- [37] G. Lan, "Policy mirror descent for reinforcement learning: Linear convergence, new sampling complexity, and generalized problem classes," *Math. Program.*, vol. 198, no. 1, pp. 1059–1106, 2023.
- [38] Y. Ding, J. Zhang, H. Lee, and J. Lavaei, "Beyond exact gradients: Convergence of stochastic soft-max policy gradient methods with entropy regularization," *IEEE Trans. Autom. Control*, vol. 70, no. 8, pp. 5129–5144, Aug. 2025.
- [39] J. Zhang et al., "On the convergence and sample efficiency of variance-reduced policy gradient method," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2021, vol. 34, pp. 2228–2240.
- [40] K. Zhang, A. Koppel, H. Zhu, and T. Basar, "Global convergence of policy gradient methods to (almost) locally optimal policies," *SIAM J. Control Optim.*, vol. 58, no. 6, pp. 3586–3612, 2020.
- [41] D. Davis, D. Drusvyatskiy, S. Kakade, and J. D. Lee, "Stochastic subgradient method converges on tame functions," *Found. Computat. Math.*, vol. 20, no. 1, pp. 119–154, 2020.
- [42] D. Lee, N. He, P. Kamalaruban, and V. Cevher, "Optimization for reinforcement learning: From a single agent to cooperative agents," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 123–135, May 2020.
- [43] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, no. 3, pp. 229–256, 1992.
- [44] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2015, *arXiv:1506.02438*.
- [45] E. Greensmith, P. L. Bartlett, and J. Baxter, "Variance reduction techniques for gradient estimates in reinforcement learning," *J. Mach. Learn. Res.*, vol. 5, no. 9, 2004, pp. 1471–1530.
- [46] V. Konda and J. Tsitsiklis, "Actor-critic algorithms," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 1999, vol. 12, pp. 1008–1014.
- [47] S. M. Kakade, "A natural policy gradient," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2001, vol. 14, pp. 1531–1538.
- [48] S. Kakade and J. Langford, "Approximately optimal approximate reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2002, pp. 267–274.
- [49] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 1889–1897.
- [50] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [51] R. Bro, "PARAFAC. Tutorial and applications," *Chemometrics Intell. Lab. Syst.*, vol. 38, no. 2, pp. 149–171, 1997.
- [52] S. Rozada and A. G. Marques, "Tensor low-rank approximation of finite-horizon value functions," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2024, pp. 5975–5979.
- [53] S. Amari, *Differential-Geometrical Methods in Statistics*, vol. 28. Berlin, Heidelberg: Springer-Verlag, 2012.
- [54] D. Davis and D. Drusvyatskiy, "Stochastic model-based minimization of weakly convex functions," *SIAM J. Optim.*, vol. 29, no. 1, pp. 207–239, 2019.
- [55] S. Rozada, "Online code repository: Tensor low-rank approximation for policy-gradient methods in reinforcement learning." GitHub. Accessed: Oct. 27, 2025. [Online]. Available: <https://github.com/sergiorozada12/tensor-low-rank-pg>
- [56] G. Brockman et al., "OpenAI gym," 2016, *arXiv:1606.01540*.
- [57] "Goddard's rocket problem," GitHub. Accessed: Oct. 25, 2025. [Online]. Available: <https://github.com/osannolik/gym-goddard>
- [58] M. D. Buhmann, "Radial basis functions," *Acta Numerica*, vol. 9, pp. 1–38, Jan. 2000.



Sergio Rozada (Student Member, IEEE) received the B.Sc. degree in electronics and automation engineering from the University of Oviedo, Spain, in 2015, the M.Sc. degree (with distinction) in data science from City, University of London, London, U.K., in 2018, and the Ph.D. degree in electrical engineering from King Juan Carlos University (URJC), Madrid, Spain, in 2025. In 2025, he joined the Department of Signal Theory and Communications, URJC as an Assistant Professor, where he currently carries out his research and teaching activities. He has also held a Research Appointment with the Department of Electrical and Systems Engineering, University of Pennsylvania, Philadelphia, PA, USA. His research interests include dynamical systems and reinforcement learning, with broader interests in decision-making and sequential, nonlinear, and stochastic optimization. His doctoral dissertation received the Young Researchers and Innovators Award for the best engineering dissertation at URJC in 2025.



Hoi-To Wai (Member, IEEE) received the Ph.D. degree in electrical engineering from Arizona State University (ASU), Tempe, AZ, USA, in 2017, the B.Eng. (with First Class Hons.) and M.Phil. degrees in electronic engineering from The Chinese University of Hong Kong (CUHK), Hong Kong, in 2010 and 2012, respectively. He is currently an Assistant Professor with the Department of Systems Engineering and Engineering Management, CUHK. He has held Research Positions with ASU, University of California, Davis, CA, USA; Telecom ParisTech, Paris, France; Ecole Polytechnique, Palaiseau, France; LIDS, Massachusetts Institute of Technology, Cambridge, MA, USA. His research interests include signal processing, machine learning, and distributed optimization, with a focus of their applications to network science. His dissertation was the recipient of the 2017's Dean's Dissertation Award from the Ira A. Fulton Schools of Engineering of ASU, and Best Student Paper Award at ICASSP 2018. He is currently an Associate Editor for IEEE TRANSACTIONS ON SIGNAL PROCESSING and Elseviers *Signal Processing*.



Antonio G. Marques (Senior Member, IEEE) received the telecommunications engineering degree and the Ph.D. degree (both with Highest Hons.), from Carlos III University of Madrid, Spain, in 2002 and 2007, respectively. In 2007, he was a Faculty with the Department of Signal Theory and Communications, King Juan Carlos University, Madrid, Spain, where he currently develops his Research and Teaching Activities as a Full Professor and Director of the Data Science and Signal Processing for Networks Research Group. From 2005 to 2015, he held different Visiting Positions with the University of Minnesota, Minneapolis. In 2015, 2016 and 2017 he was a Visitor Scholar with the University of Pennsylvania, Philadelphia. Since 2025, he serves as a Co-Director of the ELLIS Madrid Unit. His research interests include signal processing, machine learning, data science and artificial intelligence over graphs, and nonlinear and stochastic optimization of wireless and transportation networks. He has served the IEEE in a number of posts, including as an associate Editor and the Technical/General Chair of different conferences and, he is currently a Senior Area Editor of IEEE TRANSACTIONS ON SIGNAL PROCESS. His work has been awarded in several journals, conferences and workshops, with recent ones including the IEEE SPS IEEE Y.A. Best Paper Award 2020, and the IEEE SPS IEEE Best Paper Award 2024. He is a recipient of the 2020 EURASIP Early Career Award and a member of EURASIP and the ELLIS society.